بِسْمِ اللَّهِ الرَّحْمَٰنِ الرَّحِيمِ

LIVE ONLINE COURSE | COHORT FALL 2026

AI & Machine Learning for Civil Engineers

SESSION 02 · DATA BEFORE ALGORITHMS

From Raw Engineering Data to an ML-Ready Dataset

Can we trust our engineering data before giving it to a model?

- DATA QUALITY
- EXPLORATION
- PREPARATION
- EVALUATION SETUP



Dr. Abdul Waris
Research Associate · Universität Siegen, Germany

Technology partner QUDELTA STUDIOS

WHERE WE LEFT OFF · SESSION 01

Last time: X, y and one engineering workflow

WHERE WE LEFT OFF · SESSION 01

Last time: X, y and one engineering workflow

X

Features

The inputs we give the model: the mix and its age.

`cement, water, age …`

y

Target

What we want to predict. The target decides the ML problem.

`strength (MPa)`

WHERE WE LEFT OFF · SESSION 01

Last time: X, y and one engineering workflow

X

Features

The inputs we give the model: the mix and its age.

`cement, water, age …`

y

Target

What we want to predict. The target decides the ML problem.

`strength (MPa)`

One workflow

Problem, data, Python, model, decision: the same chain for every application.

`problem → … → decision`

A notebook

We loaded 1,030 lab tests with pandas and drew our first plot.

`pd.read_csv("concrete.csv")`

WHERE WE LEFT OFF · SESSION 01

Last time: X, y and one engineering workflow

X

Features

The inputs we give the model: the mix and its age.

`cement, water, age …`

y

Target

What we want to predict. The target decides the ML problem.

`strength (MPa)`

One workflow

Problem, data, Python, model, decision: the same chain for every application.

`problem → … → decision`

A notebook

We loaded 1,030 lab tests with pandas and drew our first plot.

`pd.read_csv("concrete.csv")`

Today we stop assuming the table is ready. We ask whether it deserves a model.

THE COURSE AT A GLANCE

Eight sessions. One engineering workflow.

PART 1 · FOUNDATIONS

01 From Engineering Problems to Python DONE

AI/ML basics, Google Colab, NumPy & pandas

02 From Raw Engineering Data to an ML-Ready Dataset TODAY

EDA, data quality, outliers, train/test split, leakage

03 Your First Engineering Prediction Model

Linear regression, error metrics, cross-validation

04 Nonlinear Engineering Problems: Trees & Ensembles

Decision trees, Random Forest, boosting

PART 2 · PREDICTION TO DECISION

05 From Prediction to Engineering Decision: Classification

Logistic regression, confusion matrix, precision & recall

06 Artificial Neural Networks

Architecture, activations, training, overfitting

07 Can We Trust Our ML Model?

Feature importance, SHAP, engineering judgement

Instructor's research area

08 From Prediction to Probability

Monte Carlo, ML surrogates, probability of failure

Instructor's research area

TODAY · SESSION 02

From raw engineering data to an ML-ready dataset

TODAY · SESSION 02

From raw engineering data to an ML-ready dataset

01

Know your data

What “ML-ready” means · three datasets · the first audit · plausibility



TODAY · SESSION 02

From raw engineering data to an ML-ready dataset

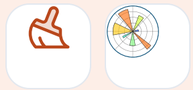
01 **Know your data**

What “ML-ready” means · three datasets · the first audit · plausibility



02 **Clean & explore**

Missing values · outliers · distributions · correlation vs causation



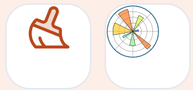
TODAY · SESSION 02

From raw engineering data to an ML-ready dataset

01 **Know your data**
What “ML-ready” means · three datasets · the first audit · plausibility



02 **Clean & explore**
Missing values · outliers · distributions · correlation vs causation



03 **Prepare to model**
X and y · leakage · train/test split · class balance · scaling



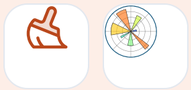
TODAY · SESSION 02

From raw engineering data to an ML-ready dataset

01 **Know your data**
What “ML-ready” means · three datasets · the first audit · plausibility



02 **Clean & explore**
Missing values · outliers · distributions · correlation vs causation



03 **Prepare to model**
X and y · leakage · train/test split · class balance · scaling

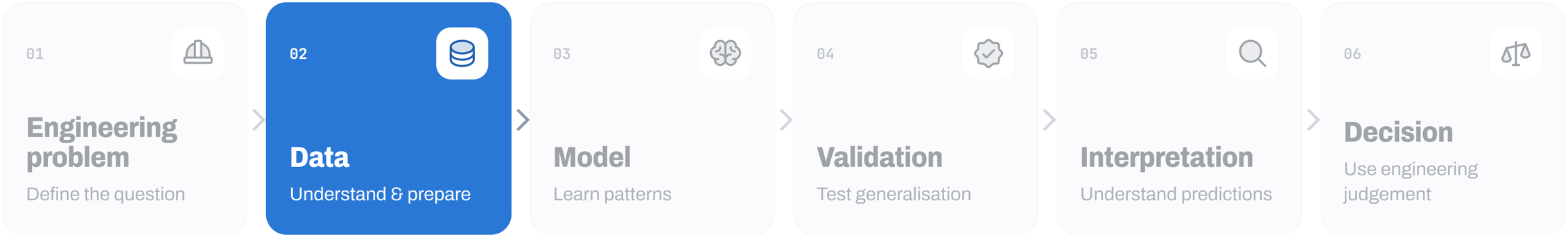


BY THE END OF TODAY YOU CAN

- Audit a dataset: size, types, missing values, duplicates and plausible ranges
- Decide what to do with missing values and outliers, and say why
- Read distributions and correlations without over-reading them
- Choose X and y, spot leakage, and split train and test reproducibly

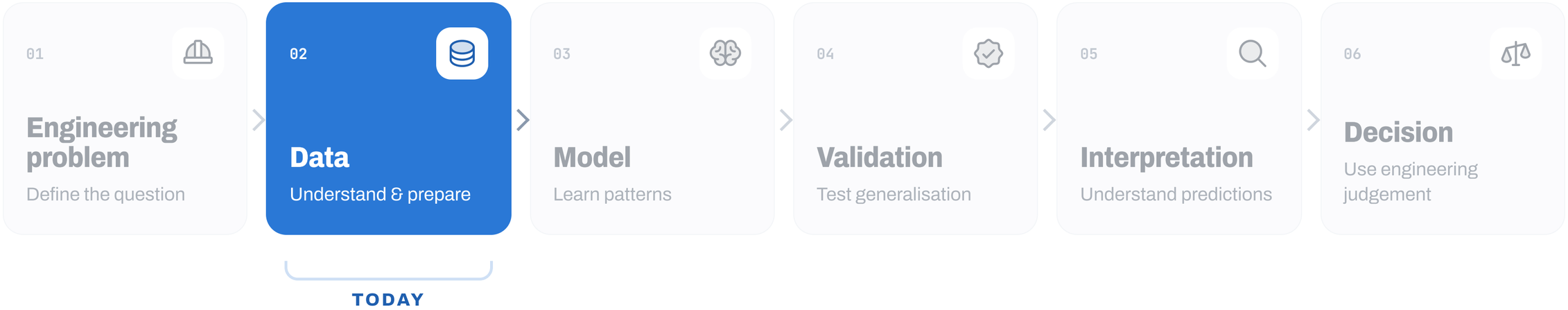
SESSION 02 · WHERE TODAY FITS

The data step decides what every later step can do



SESSION 02 · WHERE TODAY FITS

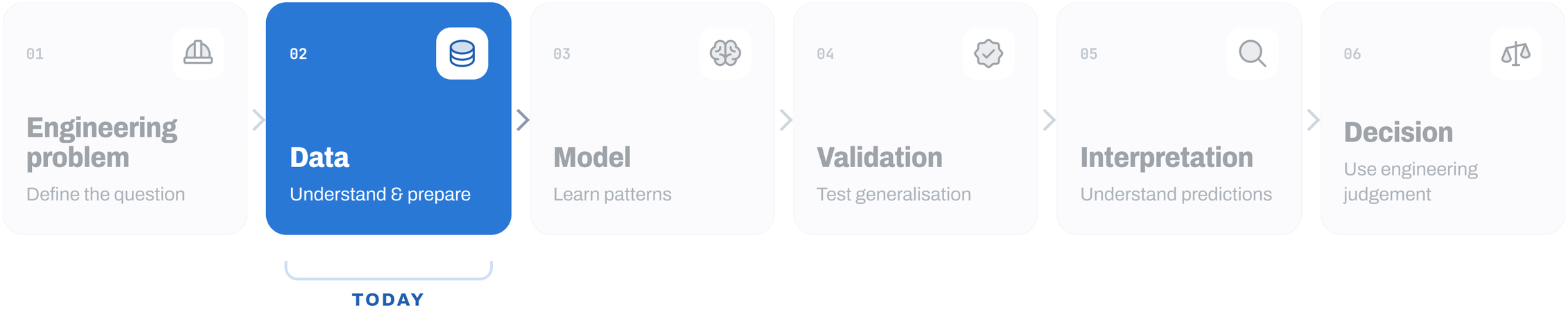
The data step decides what every later step can do



A model is only as meaningful as the problem formulation and data behind it.

SESSION 02 · WHERE TODAY FITS

The data step decides what every later step can do

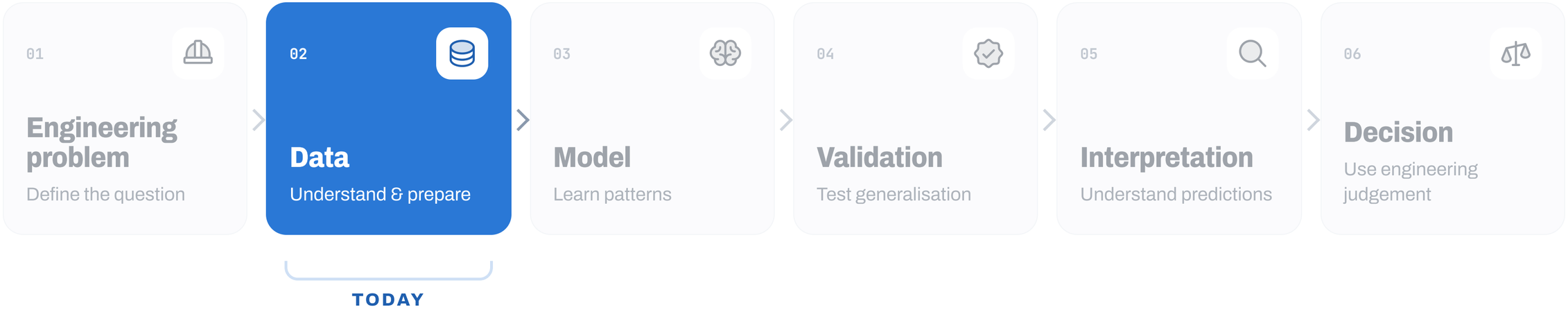


A model is only as meaningful as the problem formulation and data behind it.

Wrong units
→ a model that learns the wrong physics

SESSION 02 · WHERE TODAY FITS

The data step decides what every later step can do



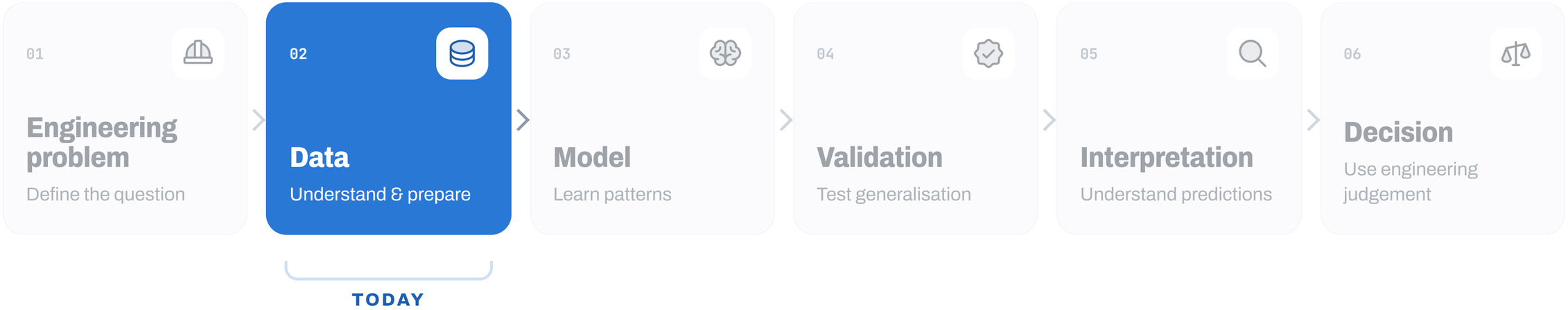
A model is only as meaningful as the problem formulation and data behind it.

Wrong units
→ a model that learns the wrong physics

Unexamined gaps
→ a model blind to unusual conditions

SESSION 02 · WHERE TODAY FITS

The data step decides what every later step can do



A model is only as meaningful as the problem formulation and data behind it.

Wrong units
→ a model that learns the wrong physics

Unexamined gaps
→ a model blind to unusual conditions

Leaked information
→ test scores that never return in practice

01

a What “ML-ready” means

b A dataset is more than a table

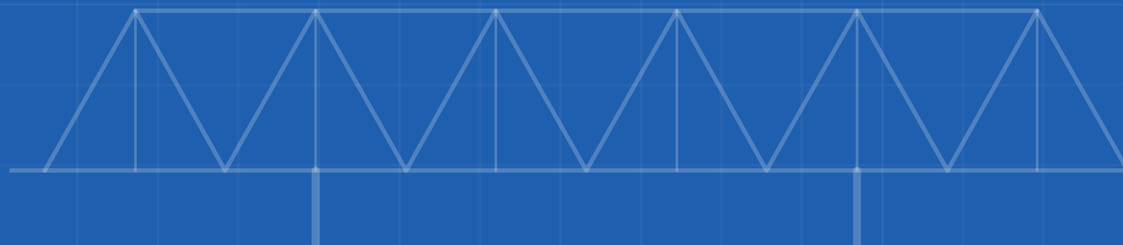
c Three datasets, three lessons

d The first audit

e Is it physically plausible?

PART 01 · DATA QUALITY

Know your data before you trust it



SIX QUESTIONS BEFORE ANY MODEL

What does “ML-ready” actually mean?

SIX QUESTIONS BEFORE ANY MODEL

What does “ML-ready” actually mean?

Observations

What does each row represent?

`one mix, tested at one age`

Variables

What does every column mean?

`“ash” = fly ash, not bottom ash`

Units

Are the values physically interpretable?

`kg/m³ · days · MPa`

SIX QUESTIONS BEFORE ANY MODEL

What does “ML-ready” actually mean?

Observations

What does each row represent?

`one mix, tested at one age`

Variables

What does every column mean?

`“ash” = fly ash, not bottom ash`

Units

Are the values physically interpretable?

`kg/m³ · days · MPa`

Quality

What is missing, duplicated or unusual?

`NaN · repeated rows · extremes`

Target

What exactly are we trying to predict?

`strength at the tested age`

Availability

Would this information exist at prediction time?

`known at mix design? at day 7?`

SIX QUESTIONS BEFORE ANY MODEL

What does “ML-ready” actually mean?

Observations

What does each row represent?

`one mix, tested at one age`

Variables

What does every column mean?

`“ash” = fly ash, not bottom ash`

Units

Are the values physically interpretable?

`kg/m³ · days · MPa`

Quality

What is missing, duplicated or unusual?

`NaN · repeated rows · extremes`

Target

What exactly are we trying to predict?

`strength at the tested age`

Availability

Would this information exist at prediction time?

`known at mix design? at day 7?`

ML-ready ≠ merely clean. Clean means no errors; ML-ready means every column is understood.

ROWS, FEATURES, TARGET, UNITS

A dataset is not just a table

	cement	slag	water	age	strength
0	540.0	0.0	162.0	28	79.99
1	540.0	0.0	162.0	28	61.89
2	332.5	142.5	228.0	270	40.27
3	332.5	142.5	228.0	365	41.05

Rows: UCI Concrete Compressive Strength dataset (Yeh, 1998), first rows of the file; three of the eight inputs shown. The full dictionary has nine entries.

ROWS, FEATURES, TARGET, UNITS

A dataset is not just a table

X features: inputs used to predict

y target

	cement	slag	water	age	strength
0	540.0	0.0	162.0	28	79.99
1	540.0	0.0	162.0	28	61.89
2	332.5	142.5	228.0	270	40.27
3	332.5	142.5	228.0	365	41.05

One row = one observation: one mix, tested at one age.

Rows: UCI Concrete Compressive Strength dataset (Yeh, 1998), first rows of the file; three of the eight inputs shown. The full dictionary has nine entries.

ROWS, FEATURES, TARGET, UNITS

A dataset is not just a table

X features: inputs used to predict

y target

	cement	slag	water	age	strength
0	540.0	0.0	162.0	28	79.99
1	540.0	0.0	162.0	28	61.89
2	332.5	142.5	228.0	270	40.27
3	332.5	142.5	228.0	365	41.05

One row = one observation: one mix, tested at one age.

A DATA DICTIONARY · WRITE IT BEFORE MODELLING

COLUMN	MEANING	UNIT	ROLE
`cement`	Portland cement per cubic metre of concrete	kg/m³	X
`slag`	Ground blast-furnace slag	kg/m³	X
`water`	Mixing water	kg/m³	X
`age`	Days from casting to testing	days	X
`strength`	Compressive strength at that age	MPa	y

Rows: UCI Concrete Compressive Strength dataset (Yeh, 1998), first rows of the file; three of the eight inputs shown. The full dictionary has nine entries.

ROWS, FEATURES, TARGET, UNITS

A dataset is not just a table

X features: inputs used to predict

y target

	cement	slag	water	age	strength
0	540.0	0.0	162.0	28	79.99
1	540.0	0.0	162.0	28	61.89
2	332.5	142.5	228.0	270	40.27
3	332.5	142.5	228.0	365	41.05

One row = one observation: one mix, tested at one age.

A DATA DICTIONARY · WRITE IT BEFORE MODELLING

COLUMN	MEANING	UNIT	ROLE
`cement`	Portland cement per cubic metre of concrete	kg/m³	X
`slag`	Ground blast-furnace slag	kg/m³	X
`water`	Mixing water	kg/m³	X
`age`	Days from casting to testing	days	X
`strength`	Compressive strength at that age	MPa	y

Before modelling, understand what every column represents, and document its unit and role.

Rows: UCI Concrete Compressive Strength dataset (Yeh, 1998), first rows of the file; three of the eight inputs shown. The full dictionary has nine entries.

REAL DATA, ALL SESSION · UCI MACHINE LEARNING REPOSITORY

Three datasets, three lessons

UCI Machine Learning Repository: Concrete Compressive Strength (Yeh, 1998); Water Treatment Plant (Poch; Bejar, Cortés & Poch, 1993); Steel Plates Faults (Buscema, Terzi & Tastle, 2010).

REAL DATA, ALL SESSION · UCI MACHINE LEARNING REPOSITORY

Three datasets, three lessons

MATERIALS · REGRESSION

Concrete

1,030 mixes × 9 columns

Laboratory mixes and their compressive strength at different ages. Relatively clean.

target: strength (MPa)

TODAY'S LESSON

Variables, distributions and relationships

UCI Machine Learning Repository: Concrete Compressive Strength (Yeh, 1998); Water Treatment Plant (Poch; Bejar, Cortés & Poch, 1993); Steel Plates Faults (Buscema, Terzi & Tastle, 2010).

REAL DATA, ALL SESSION · UCI MACHINE LEARNING REPOSITORY

Three datasets, three lessons

MATERIALS · REGRESSION

Concrete

1,030 mixes × 9 columns

Laboratory mixes and their compressive strength at different ages. Relatively clean.

target: strength (MPa)

TODAY'S LESSON

Variables, distributions and relationships

WATER & ENVIRONMENT · PLANT RECORDS

Water treatment

527 days × 38 variables

Daily sensor and laboratory readings from an urban wastewater plant. Messy, like most real records.

591 values written as "?"

TODAY'S LESSON

Missing values and how to investigate them

UCI Machine Learning Repository: Concrete Compressive Strength (Yeh, 1998); Water Treatment Plant (Poch; Bejar, Cortés & Poch, 1993); Steel Plates Faults (Buscema, Terzi & Tastle, 2010).

REAL DATA, ALL SESSION · UCI MACHINE LEARNING REPOSITORY

Three datasets, three lessons

MATERIALS · REGRESSION

Concrete

1,030 mixes × 9 columns

Laboratory mixes and their compressive strength at different ages. Relatively clean.

target: strength (MPa)

TODAY'S LESSON

Variables, distributions and relationships

WATER & ENVIRONMENT · PLANT RECORDS

Water treatment

527 days × 38 variables

Daily sensor and laboratory readings from an urban wastewater plant. Messy, like most real records.

591 values written as "?"

TODAY'S LESSON

Missing values and how to investigate them

STEEL · QUALITY INSPECTION

Steel plates

1,941 plates × 27 features

Geometry and luminosity measurements of surface faults on steel plates, each labelled with one of 7 fault types.

target: fault class (7)

TODAY'S LESSON

What the target means, and class balance

UCI Machine Learning Repository: Concrete Compressive Strength (Yeh, 1998); Water Treatment Plant (Poch; Bejar, Cortés & Poch, 1993); Steel Plates Faults (Buscema, Terzi & Tastle, 2010).

REAL DATA, ALL SESSION · UCI MACHINE LEARNING REPOSITORY

Three datasets, three lessons

MATERIALS · REGRESSION

Concrete

1,030 mixes × 9 columns

Laboratory mixes and their compressive strength at different ages. Relatively clean.

target: strength (MPa)

TODAY'S LESSON

Variables, distributions and relationships

WATER & ENVIRONMENT · PLANT RECORDS

Water treatment

527 days × 38 variables

Daily sensor and laboratory readings from an urban wastewater plant. Messy, like most real records.

591 values written as "?"

TODAY'S LESSON

Missing values and how to investigate them

STEEL · QUALITY INSPECTION

Steel plates

1,941 plates × 27 features

Geometry and luminosity measurements of surface faults on steel plates, each labelled with one of 7 fault types.

target: fault class (7)

TODAY'S LESSON

What the target means, and class balance

The engineering problem changes. The data-preparation questions remain.

UCI Machine Learning Repository: Concrete Compressive Strength (Yeh, 1998); Water Treatment Plant (Poch; Bejar, Cortés & Poch, 1993); Steel Plates Faults (Buscema, Terzi & Tastle, 2010).

LIVE IN COLAB · CONCRETE DATASET

The first audit: what do we actually have?

CHECK 01

How many observations?

CHECK 02

Which variables and data types?

CHECK 03

Any missing values?

CHECK 04

Any duplicate records?

CHECK 05

Are the values plausible?

CHECK 06

What is the target?

Session02_data_preparation.ipynb audit

```
df.shape

df.dtypes.value_counts()

df.isna().sum().sum()

df.duplicated().sum()

df.describe()   # min, max, mean …
```

LIVE IN COLAB · CONCRETE DATASET

The first audit: what do we actually have?

CHECK 01

How many observations?

CHECK 02

Which variables and data types?

CHECK 03

Any missing values?

CHECK 04

Any duplicate records?

CHECK 05

Are the values plausible?

CHECK 06

What is the target?

Session02_data_preparation.ipynb audit

```
df.shape                                  (1030, 9)

df.dtypes.value_counts()

df.isna().sum().sum()

df.duplicated().sum()

df.describe()   # min, max, mean …
```

LIVE IN COLAB · CONCRETE DATASET

The first audit: what do we actually have?

CHECK 01

How many observations?

CHECK 02

Which variables and data types?

CHECK 03

Any missing values?

CHECK 04

Any duplicate records?

CHECK 05

Are the values plausible?

CHECK 06

What is the target?

Session02_data_preparation.ipynb — audit

```
df.shape                                  (1030, 9)
df.dtypes.value_counts()                  float64 8 · int64 1
df.isna().sum().sum()
df.duplicated().sum()
df.describe()   # min, max, mean …
```

LIVE IN COLAB · CONCRETE DATASET

The first audit: what do we actually have?

CHECK 01

How many observations?

CHECK 02

Which variables and data types?

CHECK 03

Any missing values?

CHECK 04

Any duplicate records?

CHECK 05

Are the values plausible?

CHECK 06

What is the target?

Session02_data_preparation.ipynb — audit

```
df.shape                                  (1030, 9)
df.dtypes.value_counts()                  float64 8 · int64 1
df.isna().sum().sum()                     0
df.duplicated().sum()
df.describe()   # min, max, mean …
```

LIVE IN COLAB · CONCRETE DATASET

The first audit: what do we actually have?

CHECK 01

How many observations?

CHECK 02

Which variables and data types?

CHECK 03

Any missing values?

CHECK 04

Any duplicate records?

CHECK 05

Are the values plausible?

CHECK 06

What is the target?

Session02_data_preparation.ipynb — audit

```
df.shape                                  (1030, 9)
df.dtypes.value_counts()                  float64 8 · int64 1
df.isna().sum().sum()                     0
df.duplicated().sum()                     25
df.describe()   # min, max, mean …
```

LIVE IN COLAB · CONCRETE DATASET

The first audit: what do we actually have?

CHECK 01

How many observations?

CHECK 02

Which variables and data types?

CHECK 03

Any missing values?

CHECK 04

Any duplicate records?

CHECK 05

Are the values plausible?

CHECK 06

What is the target?

Session02_data_preparation.ipynb — audit

```
df.shape                                  (1030, 9)
df.dtypes.value_counts()                  float64 8 · int64 1
df.isna().sum().sum()                     0
df.duplicated().sum()                     25
df.describe()   # min, max, mean …       → next slides
```

LIVE IN COLAB · CONCRETE DATASET

The first audit: what do we actually have?

CHECK 01

How many observations?

CHECK 02

Which variables and data types?

CHECK 03

Any missing values?

CHECK 04

Any duplicate records?

CHECK 05

Are the values plausible?

CHECK 06

What is the target?

Session02_data_preparation.ipynb — audit

```
df.shape                                  (1030, 9)
df.dtypes.value_counts()                  float64 8 · int64 1
df.isna().sum().sum()                     0
df.duplicated().sum()                     25
df.describe()   # min, max, mean …       → next slides
```

Even our “clean” dataset holds 25 repeated rows: 11 mixes appear two to four times with identical strengths.

TWO KINDS OF CHECKS

Data quality has two sides

TWO KINDS OF CHECKS

Data quality has two sides

Engineering checks

Units and dimensions

Physically plausible ranges

Measurement context

How the data were generated

`Is a w/c of 1.88 a real mix?`

TWO KINDS OF CHECKS

Data quality has two sides

Engineering checks

- Units and dimensions
- Physically plausible ranges
- Measurement context
- How the data were generated

`Is a w/c of 1.88 a real mix?`

+

Computational checks

- Missing values
- Duplicate rows
- Data types
- Invalid entries

`df.isna() · df.duplicated() · df.dtypes`

TWO KINDS OF CHECKS

Data quality has two sides

Engineering checks

- Units and dimensions
- Physically plausible ranges
- Measurement context
- How the data were generated

`Is a w/c of 1.88 a real mix?`

+

Computational checks

- Missing values
- Duplicate rows
- Data types
- Invalid entries

`df.isna() · df.duplicated() · df.dtypes`

A dataset can be computationally clean and physically wrong.

ENGINEERING CHECKS, ON REAL NUMBERS · DF.DESCRIBE()

Is every value physically plausible?

CHECK	IN THE DATA	ENGINEERING READING	VERDICT

Data: UCI Concrete Compressive Strength (Yeh, 1998). Water–binder ratio = water / (cement + slag + fly ash). The last row is a hypothetical error, not in the dataset.

ENGINEERING CHECKS, ON REAL NUMBERS · DF.DESCRIBE()

Is every value physically plausible?

CHECK	IN THE DATA	ENGINEERING READING	VERDICT
Water–cement ratio	`0.27 – 1.88`	High values belong to slag and fly-ash mixes: w/c ignores those binders. Water–binder ratio is 0.24 – 0.90.	Plausible, explained

Data: UCI Concrete Compressive Strength (Yeh, 1998). Water–binder ratio = water / (cement + slag + fly ash). The last row is a hypothetical error, not in the dataset.

ENGINEERING CHECKS, ON REAL NUMBERS · DF.DESCRIBE()

Is every value physically plausible?

CHECK	IN THE DATA	ENGINEERING READING	VERDICT
Water–cement ratio	`0.27 – 1.88`	High values belong to slag and fly-ash mixes: w/c ignores those binders. Water–binder ratio is 0.24 – 0.90.	Plausible, explained
Sum of ingredients	`2,195 – 2,551`	In kg/m^3: close to the unit weight expected of normal-weight concrete.	Plausible

Data: UCI Concrete Compressive Strength (Yeh, 1998). Water–binder ratio = water / (cement + slag + fly ash). The last row is a hypothetical error, not in the dataset.

ENGINEERING CHECKS, ON REAL NUMBERS · DF.DESCRIBE()

Is every value physically plausible?

CHECK	IN THE DATA	ENGINEERING READING	VERDICT
Water–cement ratio	`0.27 – 1.88`	High values belong to slag and fly-ash mixes: w/c ignores those binders. Water–binder ratio is 0.24 – 0.90.	Plausible, explained
Sum of ingredients	`2,195 – 2,551`	In kg/m^3: close to the unit weight expected of normal-weight concrete.	Plausible
Age at testing	`1 – 365 days`	Standard test ages: 3, 7, 28, 56, 90 days and long-term tests.	Plausible

Data: UCI Concrete Compressive Strength (Yeh, 1998). Water–binder ratio = water / (cement + slag + fly ash). The last row is a hypothetical error, not in the dataset.

ENGINEERING CHECKS, ON REAL NUMBERS · DF.DESCRIBE()

Is every value physically plausible?

CHECK	IN THE DATA	ENGINEERING READING	VERDICT
Water–cement ratio	`0.27 – 1.88`	High values belong to slag and fly-ash mixes: w/c ignores those binders. Water–binder ratio is 0.24 – 0.90.	Plausible, explained
Sum of ingredients	`2,195 – 2,551`	In kg/m^3: close to the unit weight expected of normal-weight concrete.	Plausible
Age at testing	`1 – 365 days`	Standard test ages: 3, 7, 28, 56, 90 days and long-term tests.	Plausible
Compressive strength	`2.3 – 82.6 MPa`	From very young concrete to high-strength mixes.	Plausible

Data: UCI Concrete Compressive Strength (Yeh, 1998). Water–binder ratio = water / (cement + slag + fly ash). The last row is a hypothetical error, not in the dataset.

ENGINEERING CHECKS, ON REAL NUMBERS · DF.DESCRIBE()

Is every value physically plausible?

CHECK	IN THE DATA	ENGINEERING READING	VERDICT
Water–cement ratio	`0.27 – 1.88`	High values belong to slag and fly-ash mixes: w/c ignores those binders. Water–binder ratio is 0.24 – 0.90.	Plausible, explained
Sum of ingredients	`2,195 – 2,551`	In kg/m^3: close to the unit weight expected of normal-weight concrete.	Plausible
Age at testing	`1 – 365 days`	Standard test ages: 3, 7, 28, 56, 90 days and long-term tests.	Plausible
Compressive strength	`2.3 – 82.6 MPa`	From very young concrete to high-strength mixes.	Plausible
What we would catch	`cement = 5400`	Ten times too large: a unit slip or a misplaced decimal point. Code runs; physics fails.	Investigate

Data: UCI Concrete Compressive Strength (Yeh, 1998). Water–binder ratio = water / (cement + slag + fly ash). The last row is a hypothetical error, not in the dataset.

ENGINEERING CHECKS, ON REAL NUMBERS · DF.DESCRIBE()

Is every value physically plausible?

CHECK	IN THE DATA	ENGINEERING READING	VERDICT
Water–cement ratio	`0.27 – 1.88`	High values belong to slag and fly-ash mixes: w/c ignores those binders. Water–binder ratio is 0.24 – 0.90.	Plausible, explained
Sum of ingredients	`2,195 – 2,551`	In kg/m^3: close to the unit weight expected of normal-weight concrete.	Plausible
Age at testing	`1 – 365 days`	Standard test ages: 3, 7, 28, 56, 90 days and long-term tests.	Plausible
Compressive strength	`2.3 – 82.6 MPa`	From very young concrete to high-strength mixes.	Plausible
What we would catch	`cement = 5400`	Ten times too large: a unit slip or a misplaced decimal point. Code runs; physics fails.	Investigate

Computers check types. Engineers check physics.

Data: UCI Concrete Compressive Strength (Yeh, 1998). Water–binder ratio = water / (cement + slag + fly ash). The last row is a hypothetical error, not in the dataset.

02

a Missing values: detect, investigate, decide

b Missingness as an engineering clue

c Outliers: unusual is not wrong

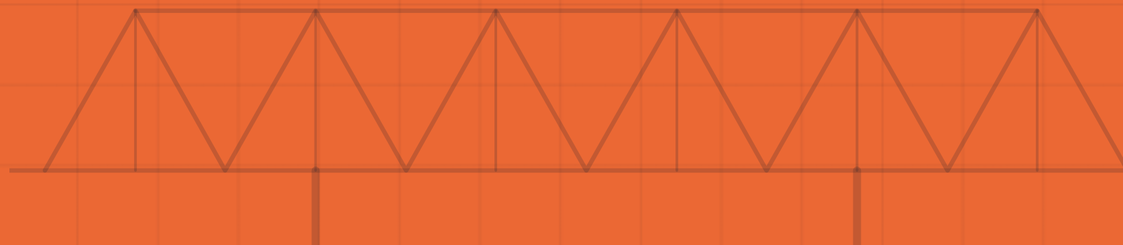
d Explore distributions

e Correlation is not causation



PART 02 · EXPLORATION

Clean and explore, with engineering eyes



MISSING DATA

“What is missing?” is not enough

MISSING DATA

“What is missing?” is not enough

1

Detect

Where are the missing values? How many? Are they concentrated in particular variables or periods?

```
df.isna().sum()
```

MISSING DATA

“What is missing?” is not enough

1

Detect

Where are the missing values? How many? Are they concentrated in particular variables or periods?

```
df.isna().sum()
```

2

Investigate

Sensor failure? Maintenance? Transmission problem? Measurement not taken? Not applicable?

```
ask the people who measured
```

MISSING DATA

“What is missing?” is not enough

1

Detect

Where are the missing values? How many? Are they concentrated in particular variables or periods?

```
df.isna().sum()
```

2

Investigate

Sensor failure? Maintenance? Transmission problem? Measurement not taken? Not applicable?

```
ask the people who measured
```

3

Decide

Delete, impute, or use a domain-informed treatment. Choose from the context, not from habit.

```
document the choice
```

MISSING DATA

“What is missing?” is not enough

1

Detect

Where are the missing values? How many? Are they concentrated in particular variables or periods?

```
df.isna().sum()
```

2

Investigate

Sensor failure? Maintenance? Transmission problem? Measurement not taken? Not applicable?

```
ask the people who measured
```

3

Decide

Delete, impute, or use a domain-informed treatment. Choose from the context, not from habit.

```
document the choice
```

There is no universal “remove missing values” rule.

WATER TREATMENT PLANT · LIVE IN COLAB

Real engineering data are messy

RAW VALUE
"?"

REPRESENT
NaN

DETECT
Count & map

INVESTIGATE
Why missing?

DECIDE
Document

Session02_data_preparation.ipynb water

```
wt = pd.read_csv("water-treatment.data",
                 header=None, na_values="?")
wt.isna().sum().sum()              # all gaps
wt.isna().any(axis=1).sum()     # days with a gap
```

Data: Water Treatment Plant, UCI Machine Learning Repository (Poch; Bejar, Cortés & Poch, 1993). Column names are not in the file; we add them from its description.

WATER TREATMENT PLANT · LIVE IN COLAB

Real engineering data are messy

RAW VALUE
"?"

REPRESENT
NaN

DETECT
Count & map

INVESTIGATE
Why missing?

DECIDE
Document

Session02_data_preparation.ipynb water

```
wt = pd.read_csv("water-treatment.data",
                 header=None, na_values="?")
wt.isna().sum().sum()             # all gaps
wt.isna().any(axis=1).sum()     # days with a gap
```

OUTPUT

```
591
147
```

Data: Water Treatment Plant, UCI Machine Learning Repository (Poch; Bejar, Cortés & Poch, 1993). Column names are not in the file; we add them from its description.

WATER TREATMENT PLANT · LIVE IN COLAB

Real engineering data are messy

RAW VALUE "?" › REPRESENT NaN › DETECT Count & map › INVESTIGATE Why missing? › DECIDE Document

Session02_data_preparation.ipynb water

```
wt = pd.read_csv("water-treatment.data",
                 header=None, na_values="?")
wt.isna().sum().sum()              # all gaps
wt.isna().any(axis=1).sum()     # days with a gap
```

OUTPUT

```
591
147
```

591 missing values across 527 days and 38 variables

Data: Water Treatment Plant, UCI Machine Learning Repository (Poch; Bejar, Cortés & Poch, 1993). Column names are not in the file; we add them from its description.

WATER TREATMENT PLANT · LIVE IN COLAB

Real engineering data are messy

RAW VALUE "?" › REPRESENT NaN › DETECT Count & map › INVESTIGATE Why missing? › DECIDE Document

Session02_data_preparation.ipynb water

```
wt = pd.read_csv("water-treatment.data",
                 header=None, na_values="?")
wt.isna().sum().sum()              # all gaps
wt.isna().any(axis=1).sum()     # days with a gap
```

OUTPUT

```
591
147
```

591 missing values across 527 days and 38 variables

147 days with at least one gap: 28% of the record

Data: Water Treatment Plant, UCI Machine Learning Repository (Poch; Bejar, Cortés & Poch, 1993). Column names are not in the file; we add them from its description.

WATER TREATMENT PLANT · LIVE IN COLAB

Real engineering data are messy

RAW VALUE "?" › REPRESENT NaN › DETECT Count & map › INVESTIGATE Why missing? › DECIDE Document

Session02_data_preparation.ipynb water

```
wt = pd.read_csv("water-treatment.data",
                 header=None, na_values="?")
wt.isna().sum().sum()              # all gaps
wt.isna().any(axis=1).sum()     # days with a gap
```

OUTPUT

```
591
147
```

591 missing values across 527 days and 38 variables

147 days with at least one gap: 28% of the record

0 gaps in pH and conductivity on the plant inflow

Data: Water Treatment Plant, UCI Machine Learning Repository (Poch; Bejar, Cortés & Poch, 1993). Column names are not in the file; we add them from its description.

WATER TREATMENT PLANT · LIVE IN COLAB

Real engineering data are messy

RAW VALUE "?" › REPRESENT NaN › DETECT Count & map › INVESTIGATE Why missing? › DECIDE Document

Session02_data_preparation.ipynb water

```
wt = pd.read_csv("water-treatment.data",
                 header=None, na_values="?")
wt.isna().sum().sum()              # all gaps
wt.isna().any(axis=1).sum()     # days with a gap
```

OUTPUT

```
591
147
```

591 missing values across 527 days and 38 variables

147 days with at least one gap: 28% of the record

0 gaps in pH and conductivity on the plant inflow

Tell pandas what “missing” looks like in this file. Otherwise “?” is read as text and the column stops being numeric.

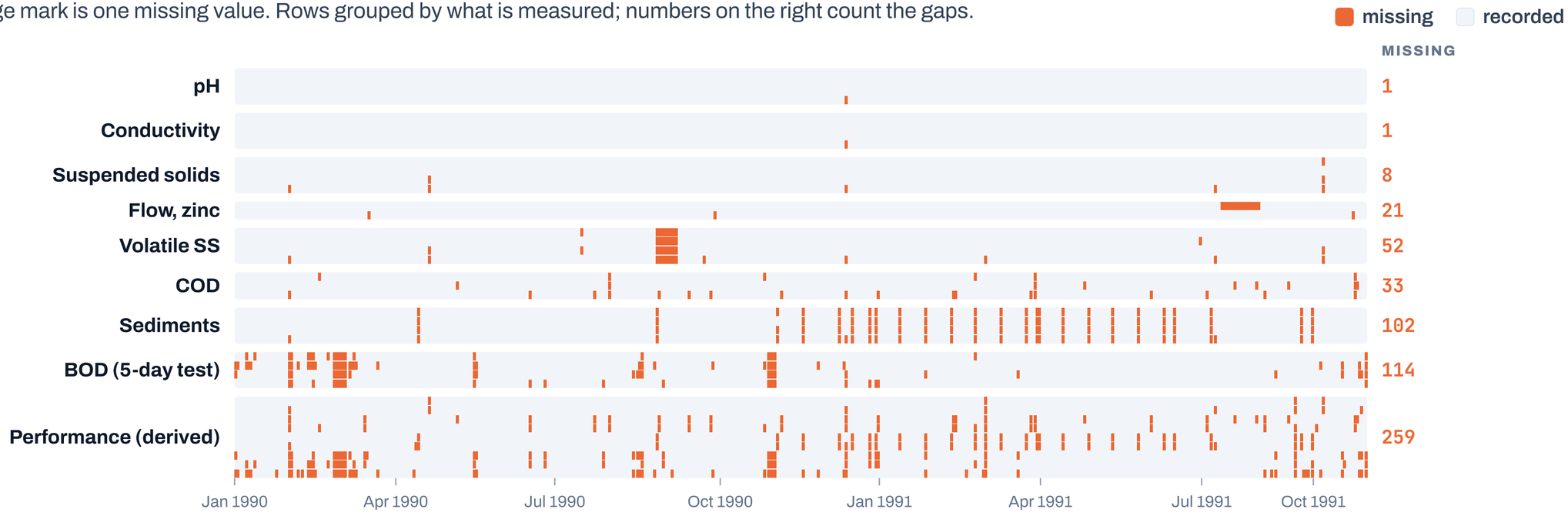
Data: Water Treatment Plant, UCI Machine Learning Repository (Poch; Bejar, Cortés & Poch, 1993). Column names are not in the file; we add them from its description.

DETECT · 591 GAPS, MAPPED DAY BY DAY

Missingness follows the measurement method

Water treatment plant · 527 days · 38 variables in 9 groups

Each orange mark is one missing value. Rows grouped by what is measured; numbers on the right count the gaps.



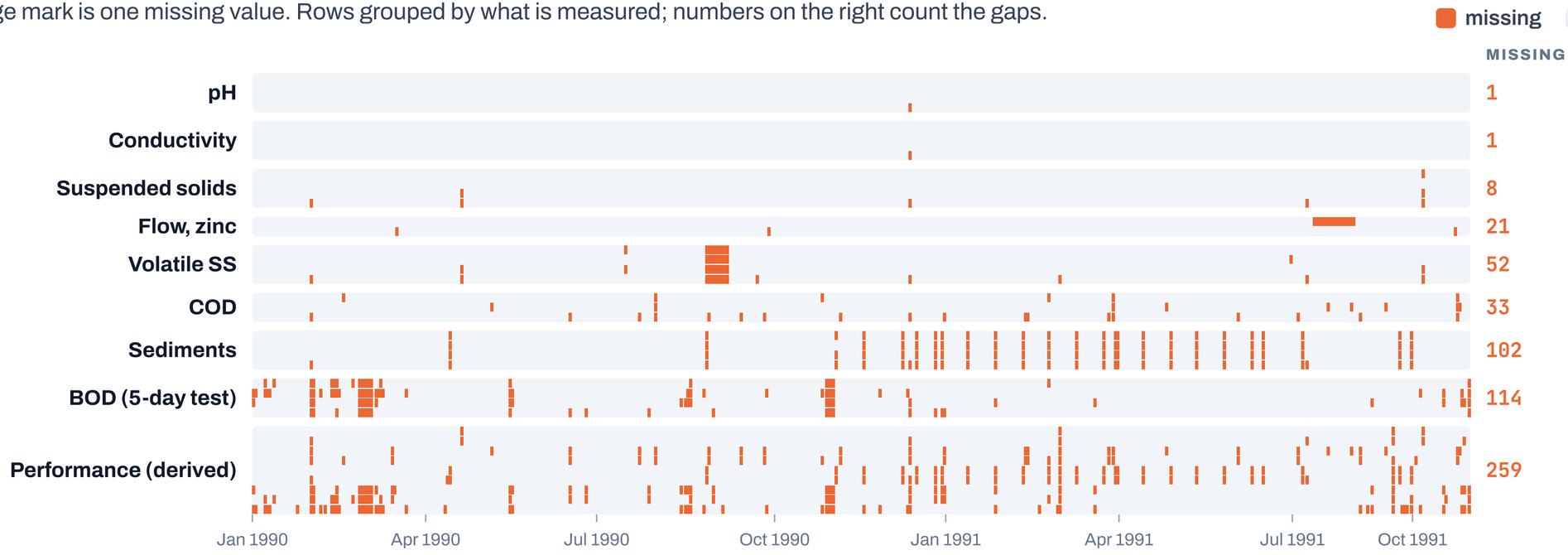
Data: UCI Water Treatment Plant. Rows sorted by date (the file stores months out of order). BOD = biological oxygen demand; COD = chemical oxygen demand; SS = suspended solids. Performance columns are removal ratios between plant stages.

DETECT · 591 GAPS, MAPPED DAY BY DAY

Missingness follows the measurement method

Water treatment plant · 527 days · 38 variables in 9 groups

Each orange mark is one missing value. Rows grouped by what is measured; numbers on the right count the gaps.



Online sensors (pH, conductivity) almost never miss. The 5-day BOD lab test, and the performance columns calculated from it, carry most gaps.

Data: UCI Water Treatment Plant. Rows sorted by date (the file stores months out of order). BOD = biological oxygen demand; COD = chemical oxygen demand; SS = suspended solids. Performance columns are removal ratios between plant stages.

INVESTIGATE

Missingness can be an engineering clue

Suppose a sensor has many missing values during one operating period.
Why?

INVESTIGATE

Missingness can be an engineering clue

Suppose a sensor has many missing values during one operating period. Why?

Equipment downtime
The system was not operating.

Maintenance
Measurements were intentionally paused.

INVESTIGATE

Missingness can be an engineering clue

Suppose a sensor has many missing values during one operating period. Why?

Equipment downtime
The system was not operating.

Maintenance
Measurements were intentionally paused.

Sensor malfunction
The gap itself signals a data-quality issue.

Unusual conditions
The process moved outside normal operation, e.g. a storm inflow.

INVESTIGATE

Missingness can be an engineering clue

Suppose a sensor has many missing values during one operating period. Why?

Equipment downtime
The system was not operating.

Maintenance
Measurements were intentionally paused.

Sensor malfunction
The gap itself signals a data-quality issue.

Unusual conditions
The process moved outside normal operation, e.g. a storm inflow.

A REAL CLUE IN THIS DATASET

5 of 527 days

fall on a **Saturday**. Whole days are absent from the calendar because of a sampling schedule, not a fault.

A gap in the calendar and a “?” in a cell are different kinds of missing. Both carry information.

INVESTIGATE

Missingness can be an engineering clue

Suppose a sensor has many missing values during one operating period. Why?

Equipment downtime
The system was not operating.

Maintenance
Measurements were intentionally paused.

Sensor malfunction
The gap itself signals a data-quality issue.

Unusual conditions
The process moved outside normal operation, e.g. a storm inflow.

A REAL CLUE IN THIS DATASET

5 of 527 days

fall on a **Saturday**. Whole days are absent from the calendar because of a sampling schedule, not a fault.

A gap in the calendar and a “?” in a cell are different kinds of missing. Both carry information.

Don’t erase information before understanding it.

DECIDE · ON THE WATER-TREATMENT RECORD

Every treatment of missing values has a cost

OPTION	WHAT IT DOES HERE	REASONABLE WHEN

DECIDE · ON THE WATER-TREATMENT RECORD

Every treatment of missing values has a cost

OPTION	WHAT IT DOES HERE	REASONABLE WHEN
Drop incomplete rows `wt.dropna()`	527 → 380 days: **28% of the record gone**, including unusual days.	Gaps are few and random

DECIDE · ON THE WATER-TREATMENT RECORD

Every treatment of missing values has a cost

OPTION	WHAT IT DOES HERE	REASONABLE WHEN
Drop incomplete rows `wt.dropna()`	527 → 380 days: **28% of the record gone**, including unusual days.	Gaps are few and random
Drop a column `wt.drop(columns=…)`	Loses a variable with many gaps, and everything it could tell us.	The variable is not needed or mostly empty

DECIDE · ON THE WATER-TREATMENT RECORD

Every treatment of missing values has a cost

OPTION	WHAT IT DOES HERE	REASONABLE WHEN
Drop incomplete rows `wt.dropna()`	527 → 380 days: **28% of the record gone**, including unusual days.	Gaps are few and random
Drop a column `wt.drop(columns=…)`	Loses a variable with many gaps, and everything it could tell us.	The variable is not needed or mostly empty
Fill with the median `wt.fillna(wt.median())`	Quick, but writes “typical” values into what may have been untypical days.	Gaps are short and random

DECIDE · ON THE WATER-TREATMENT RECORD

Every treatment of missing values has a cost

OPTION	WHAT IT DOES HERE	REASONABLE WHEN
Drop incomplete rows `wt.dropna()`	527 → 380 days: **28% of the record gone**, including unusual days.	Gaps are few and random
Drop a column `wt.drop(columns=…)`	Loses a variable with many gaps, and everything it could tell us.	The variable is not needed or mostly empty
Fill with the median `wt.fillna(wt.median())`	Quick, but writes “typical” values into what may have been untypical days.	Gaps are short and random
Domain-informed fill `interpolate · recompute`	Interpolate a slowly varying signal; recompute a performance ratio from its inputs.	Process knowledge supports it

DECIDE · ON THE WATER-TREATMENT RECORD

Every treatment of missing values has a cost

OPTION	WHAT IT DOES HERE	REASONABLE WHEN
Drop incomplete rows `wt.dropna()`	527 → 380 days: **28% of the record gone**, including unusual days.	Gaps are few and random
Drop a column `wt.drop(columns=…)`	Loses a variable with many gaps, and everything it could tell us.	The variable is not needed or mostly empty
Fill with the median `wt.fillna(wt.median())`	Quick, but writes “typical” values into what may have been untypical days.	Gaps are short and random
Domain-informed fill `interpolate · recompute`	Interpolate a slowly varying signal; recompute a performance ratio from its inputs.	Process knowledge supports it
Keep a “was missing” flag `wt["BOD_missing"]`	Lets a model use the fact that a value was missing.	Missingness may itself mean something

DECIDE · ON THE WATER-TREATMENT RECORD

Every treatment of missing values has a cost

OPTION	WHAT IT DOES HERE	REASONABLE WHEN
Drop incomplete rows `wt.dropna()`	527 → 380 days: **28% of the record gone**, including unusual days.	Gaps are few and random
Drop a column `wt.drop(columns=…)`	Loses a variable with many gaps, and everything it could tell us.	The variable is not needed or mostly empty
Fill with the median `wt.fillna(wt.median())`	Quick, but writes “typical” values into what may have been untypical days.	Gaps are short and random
Domain-informed fill `interpolate · recompute`	Interpolate a slowly varying signal; recompute a performance ratio from its inputs.	Process knowledge supports it
Keep a “was missing” flag `wt["BOD_missing"]`	Lets a model use the fact that a value was missing.	Missingness may itself mean something

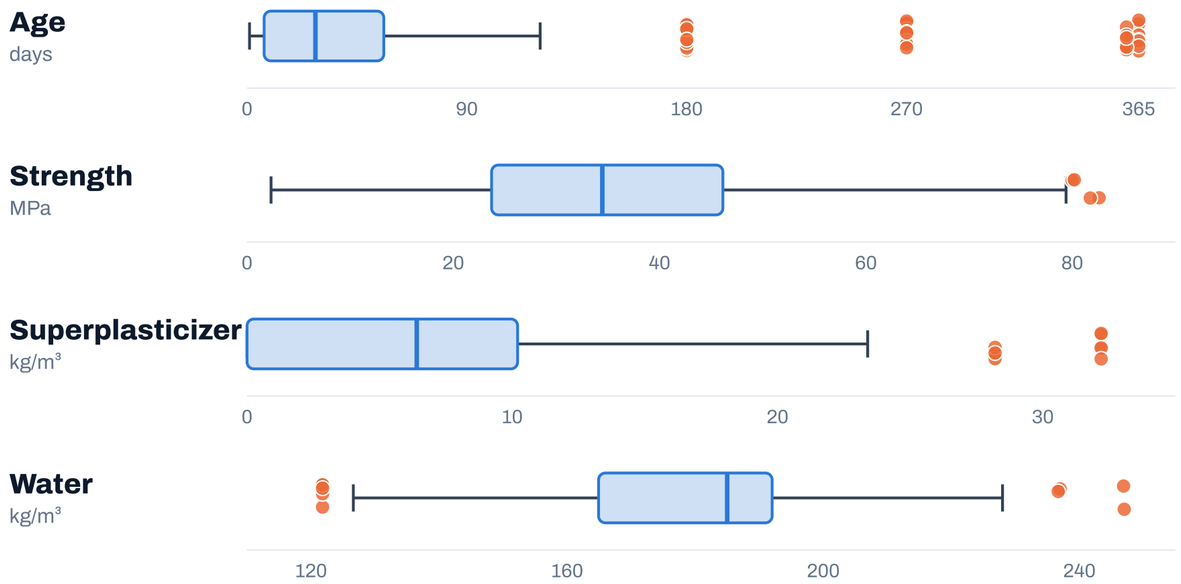
Whatever you choose, write it down: the treatment is part of the model. Learn fill values from training data only.

OUTLIERS · THE 1.5 × IQR RULE ON REAL CONCRETE DATA

Unusual does not automatically mean wrong

The rule flags 82 values in four columns

Box = middle 50% of mixes; whiskers reach 1.5 × the box width; orange points lie beyond.



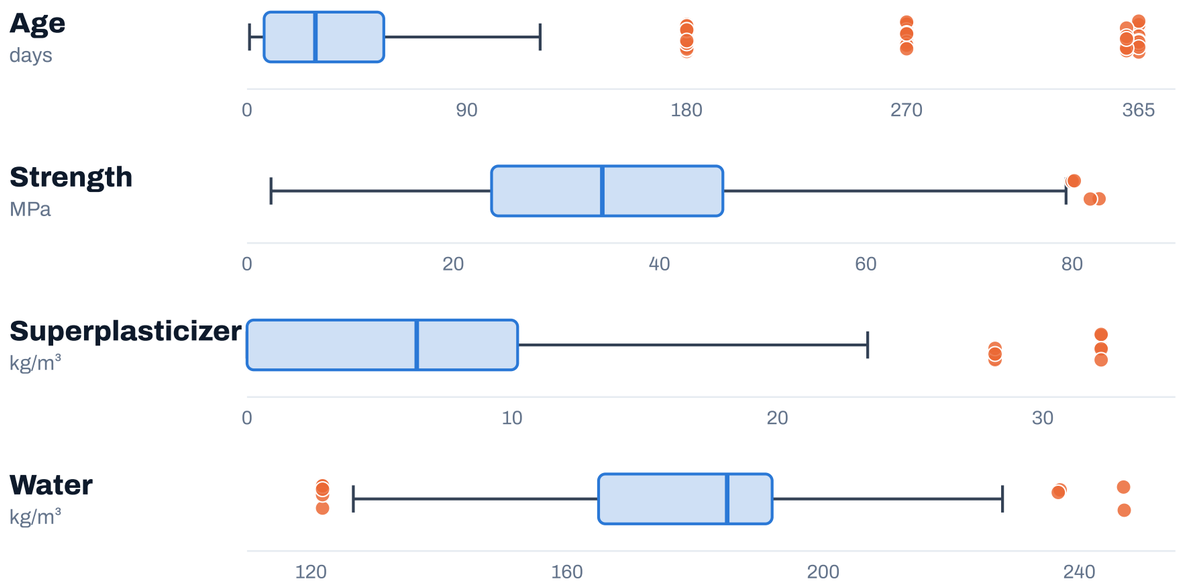
Data: UCI Concrete Compressive Strength (Yeh, 1998). IQR = interquartile range (75th minus 25th percentile).

OUTLIERS · THE 1.5 × IQR RULE ON REAL CONCRETE DATA

Unusual does not automatically mean wrong

The rule flags 82 values in four columns

Box = middle 50% of mixes; whiskers reach 1.5 × the box width; orange points lie beyond.



Investigate each flag

Measurement error

Data-entry error

Rare but legitimate condition

Extreme engineering case

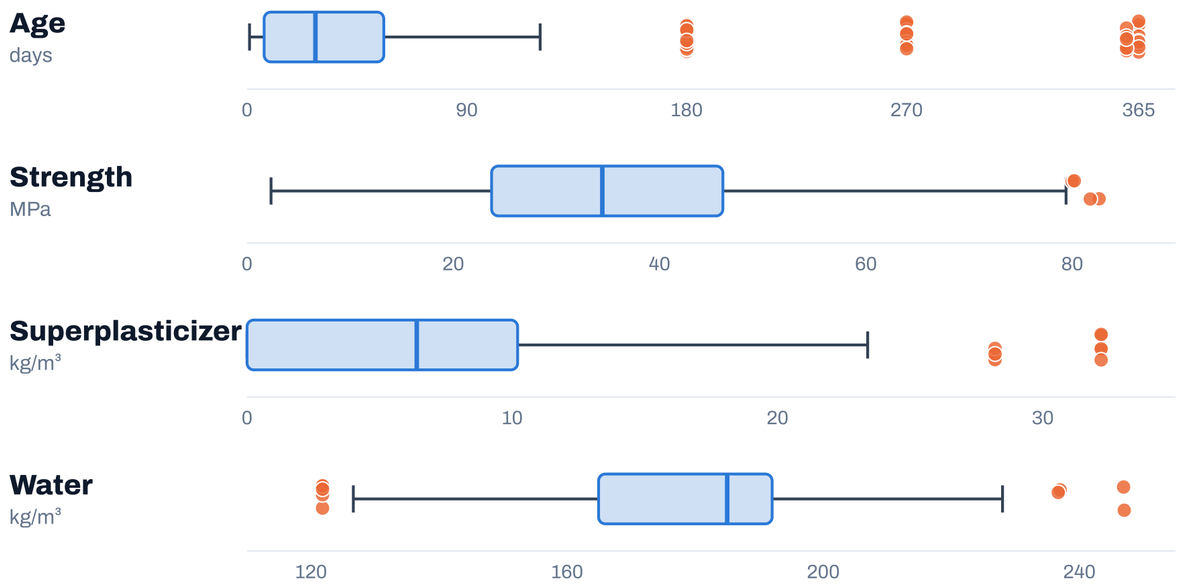
Data: UCI Concrete Compressive Strength (Yeh, 1998). IQR = interquartile range (75th minus 25th percentile).

OUTLIERS · THE 1.5 × IQR RULE ON REAL CONCRETE DATA

Unusual does not automatically mean wrong

The rule flags 82 values in four columns

Box = middle 50% of mixes; whiskers reach 1.5 × the box width; orange points lie beyond.



Investigate each flag

Measurement error

Data-entry error

Rare but legitimate condition

Extreme engineering case

HERE

All 59 “outlier” ages are real 180- to 365-day tests. Delete them and the model never learns long-term strength.

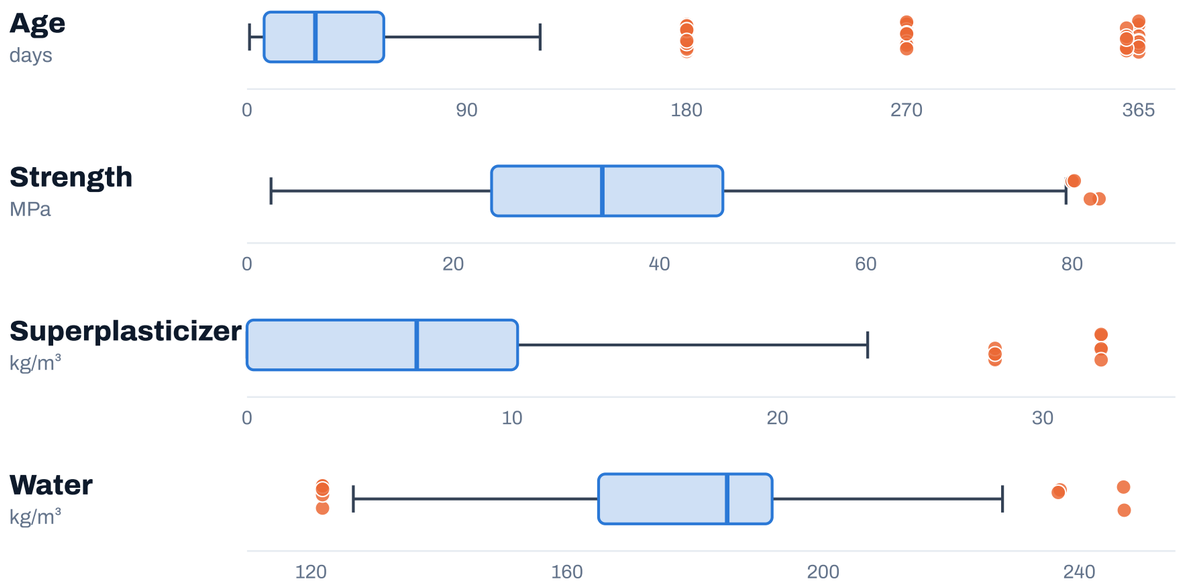
Data: UCI Concrete Compressive Strength (Yeh, 1998). IQR = interquartile range (75th minus 25th percentile).

OUTLIERS · THE 1.5 × IQR RULE ON REAL CONCRETE DATA

Unusual does not automatically mean wrong

The rule flags 82 values in four columns

Box = middle 50% of mixes; whiskers reach 1.5 × the box width; orange points lie beyond.



Investigate each flag

Measurement error

Data-entry error

Rare but legitimate condition

Extreme engineering case

HERE

All 59 “outlier” ages are real 180- to 365-day tests. Delete them and the model never learns long-term strength.

Flag → investigate → decide. Avoid deleting observations just because they are extreme.

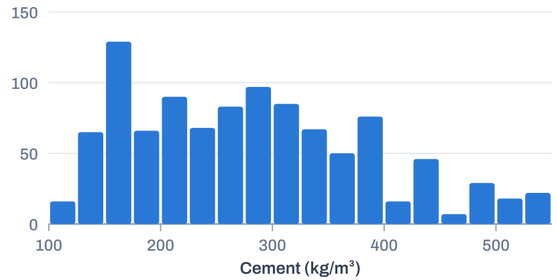
Data: UCI Concrete Compressive Strength (Yeh, 1998). IQR = interquartile range (75th minus 25th percentile).

LOOK BEFORE YOU MODEL · df.hist()

Explore before you model

LOOK BEFORE YOU MODEL · df.hist()

Explore before you model

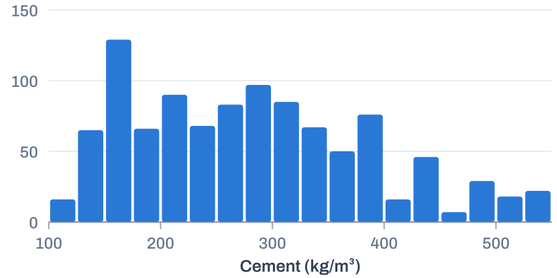


Cement

Spread from 102 to 540 kg/m³. No single “typical” mix.

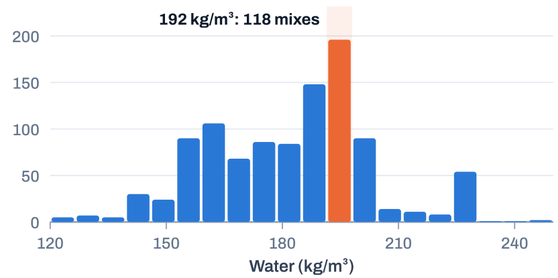
LOOK BEFORE YOU MODEL · df.hist()

Explore before you model



Cement

Spread from 102 to 540 kg/m³. No single “typical” mix.

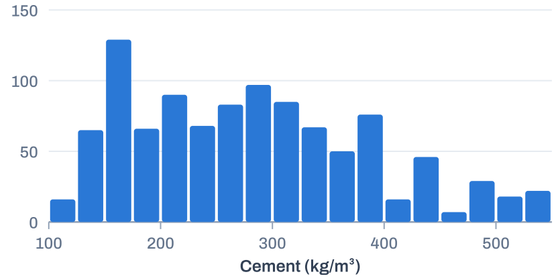


Water

Spikes at 192 and 228 kg/m³: repeated mix designs, not a natural spread.

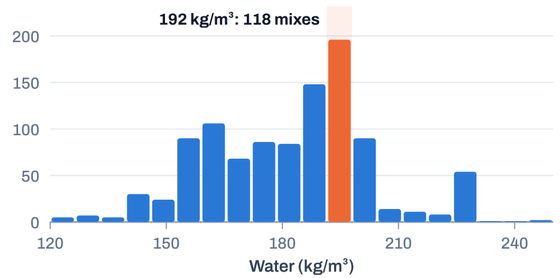
LOOK BEFORE YOU MODEL · df.hist()

Explore before you model



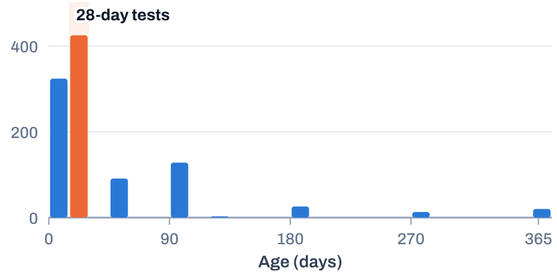
Cement

Spread from 102 to 540 kg/m³. No single “typical” mix.



Water

Spikes at 192 and 228 kg/m³: repeated mix designs, not a natural spread.

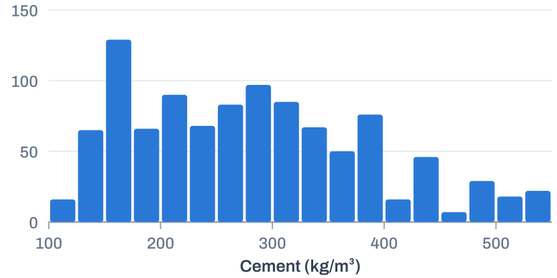


Age

A few fixed test ages, 425 of them at 28 days. Not a smooth, continuous variable.

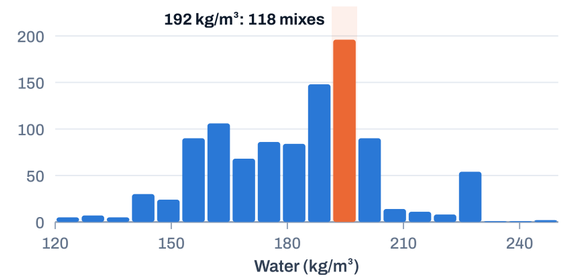
LOOK BEFORE YOU MODEL · df.hist()

Explore before you model



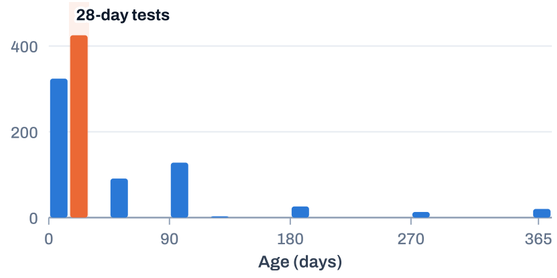
Cement

Spread from 102 to 540 kg/m³. No single “typical” mix.



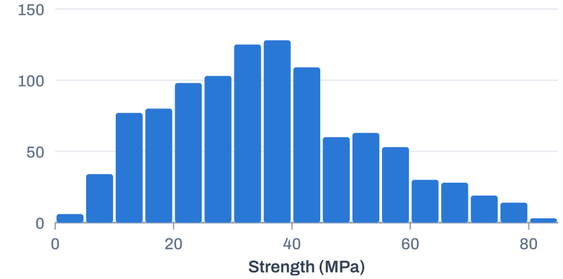
Water

Spikes at 192 and 228 kg/m³: repeated mix designs, not a natural spread.



Age

A few fixed test ages, 425 of them at 28 days. Not a smooth, continuous variable.

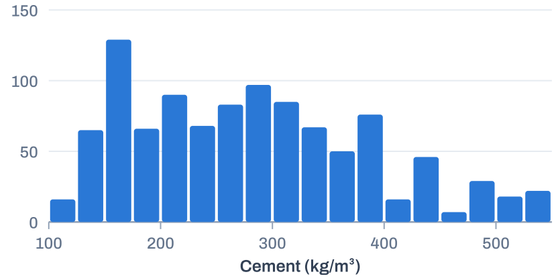


Strength

Centred near 35 MPa with a long tail to 83 MPa: the target we will predict.

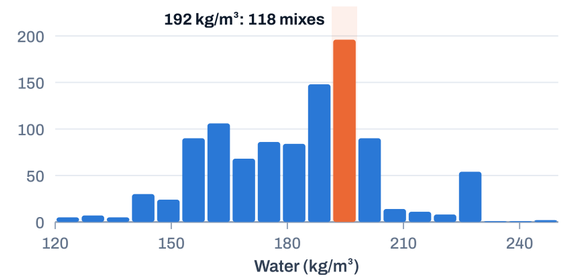
LOOK BEFORE YOU MODEL · df.hist()

Explore before you model



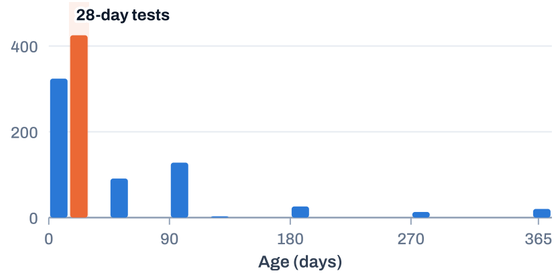
Cement

Spread from 102 to 540 kg/m³. No single “typical” mix.



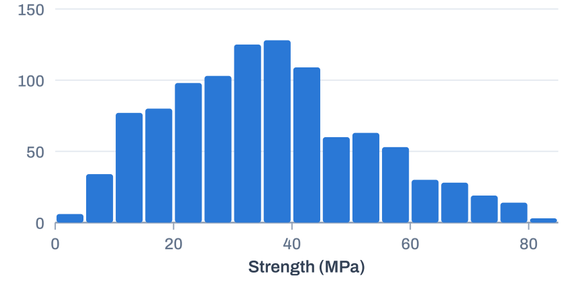
Water

Spikes at 192 and 228 kg/m³: repeated mix designs, not a natural spread.



Age

A few fixed test ages, 425 of them at 28 days. Not a smooth, continuous variable.



Strength

Centred near 35 MPa with a long tail to 83 MPa: the target we will predict.

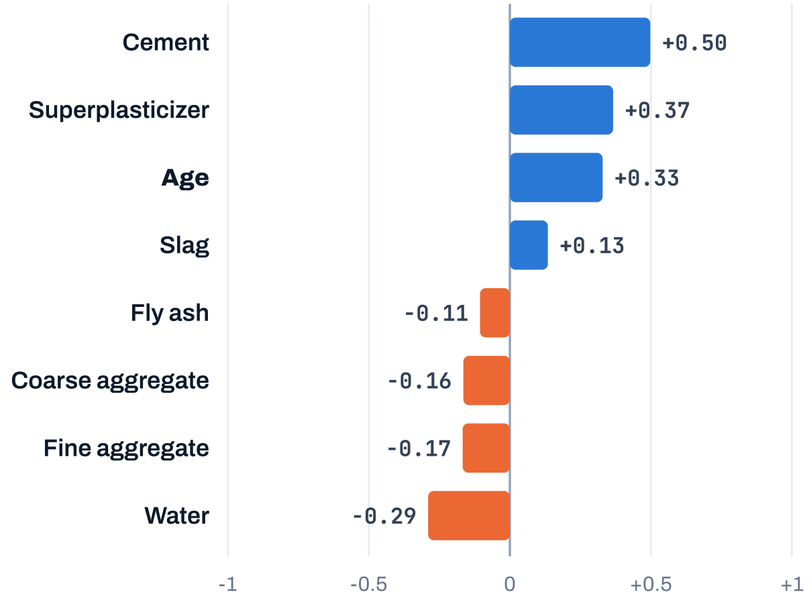
Ask: what would you notice before fitting a model? The plots are part of understanding, not decoration.

PEARSON CORRELATION WITH STRENGTH · 1,030 MIXES · df.corr()

Correlation is a clue, not an answer

Linear association of each input with strength

−1 strong negative | 0 no linear association | +1 strong positive



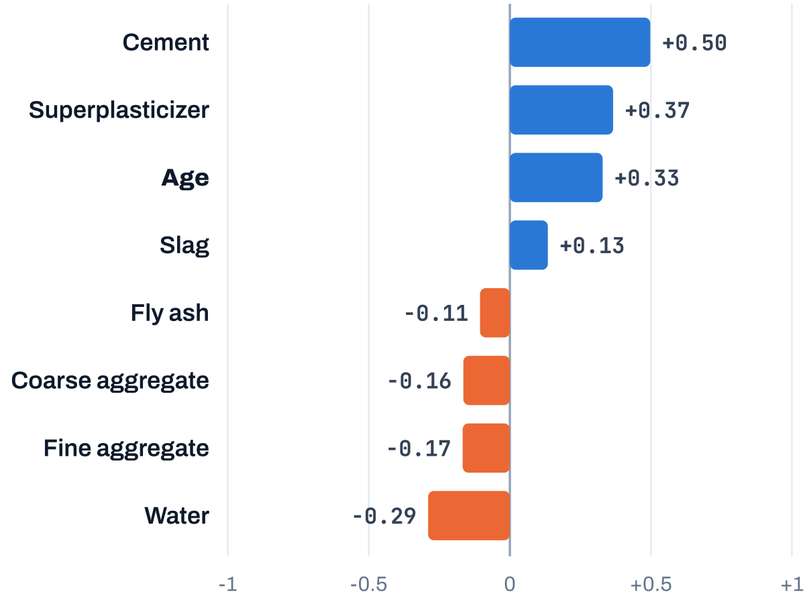
Data: UCI Concrete Compressive Strength (Yeh, 1998). Right: 520 of the 1,030 mixes shown, jittered by ±3 days; green line = median strength per group of test ages.

PEARSON CORRELATION WITH STRENGTH · 1,030 MIXES · df.corr()

Correlation is a clue, not an answer

Linear association of each input with strength

−1 strong negative | 0 no linear association | +1 strong positive



Age: r = +0.33, yet it matters a lot

Strength rises quickly, then levels off. A straight-line measure misses that shape.



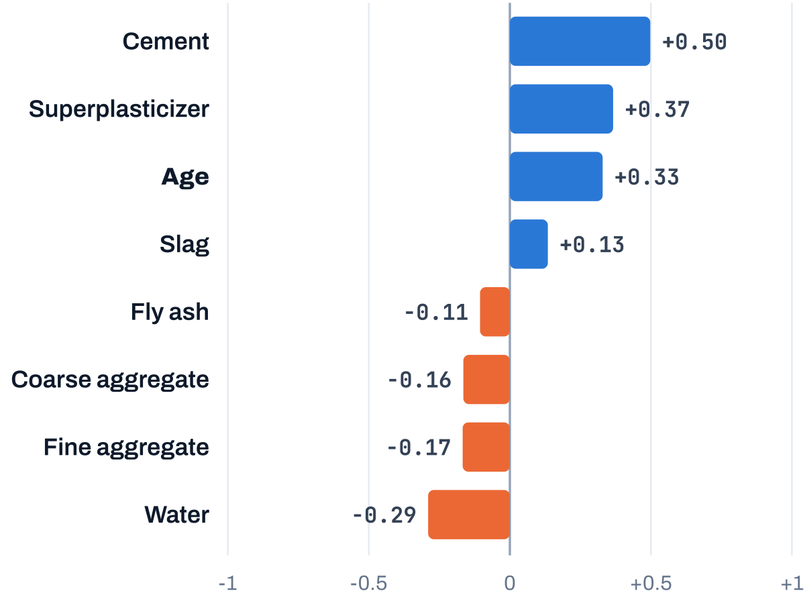
Data: UCI Concrete Compressive Strength (Yeh, 1998). Right: 520 of the 1,030 mixes shown, jittered by ±3 days; green line = median strength per group of test ages.

PEARSON CORRELATION WITH STRENGTH · 1,030 MIXES · df.corr()

Correlation is a clue, not an answer

Linear association of each input with strength

−1 strong negative | 0 no linear association | +1 strong positive



Age: r = +0.33, yet it matters a lot

Strength rises quickly, then levels off. A straight-line measure misses that shape.

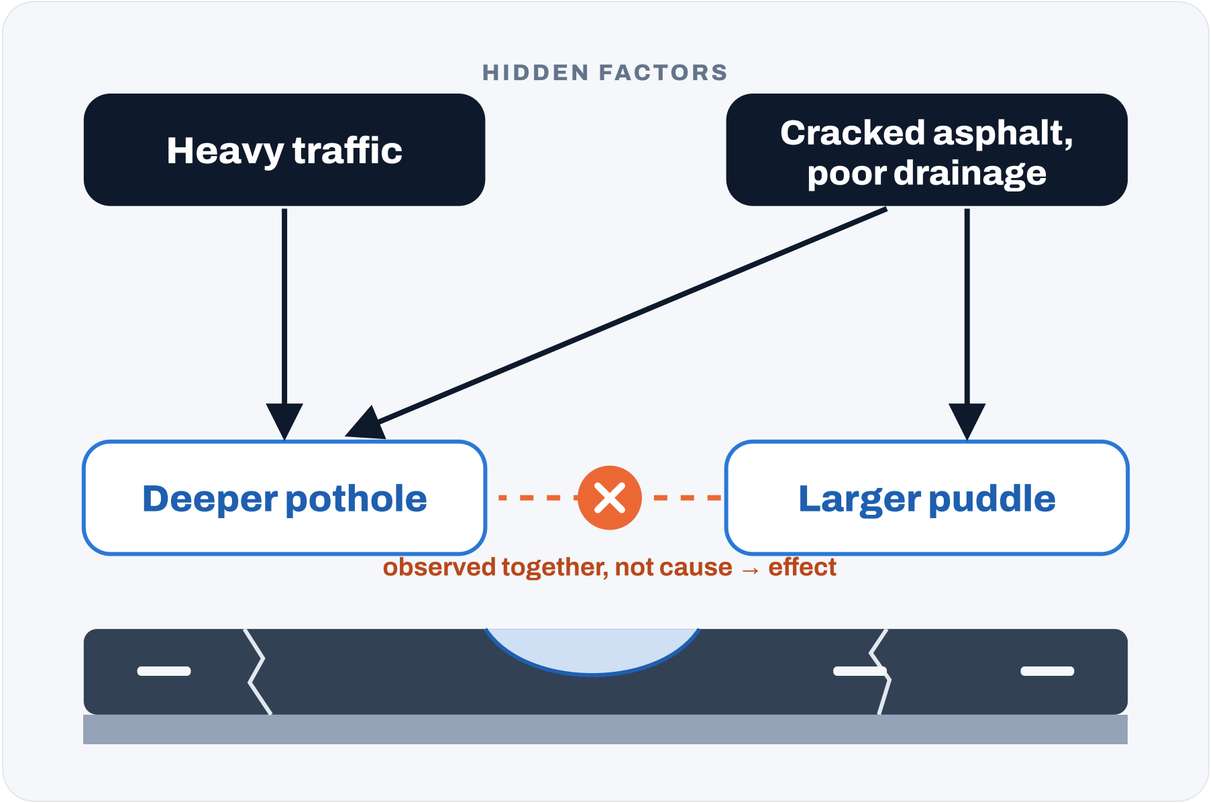


Correlation measures linear association: nonlinear relationships can be missed. And correlation ≠ causation.

Data: UCI Concrete Compressive Strength (Yeh, 1998). Right: 520 of the 1,030 mixes shown, jittered by ±3 days; green line = median strength per group of test ages.

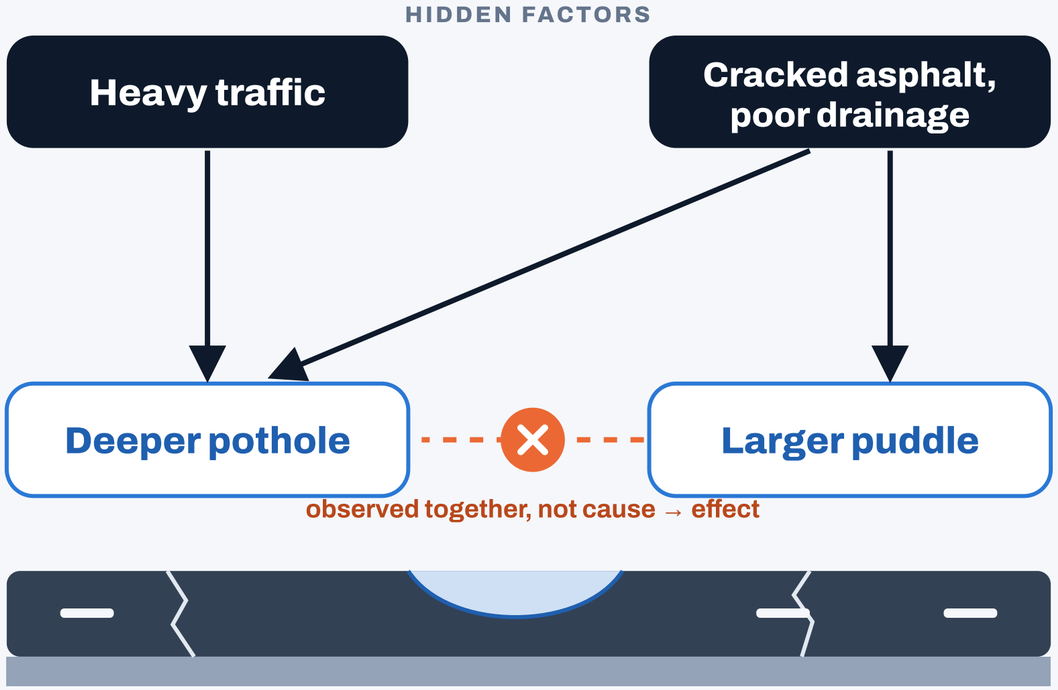
CORRELATION ≠ CAUSATION · A ROAD-ENGINEERING EXAMPLE

Do puddles cause potholes?



CORRELATION ≠ CAUSATION · A ROAD-ENGINEERING EXAMPLE

Do puddles cause potholes?

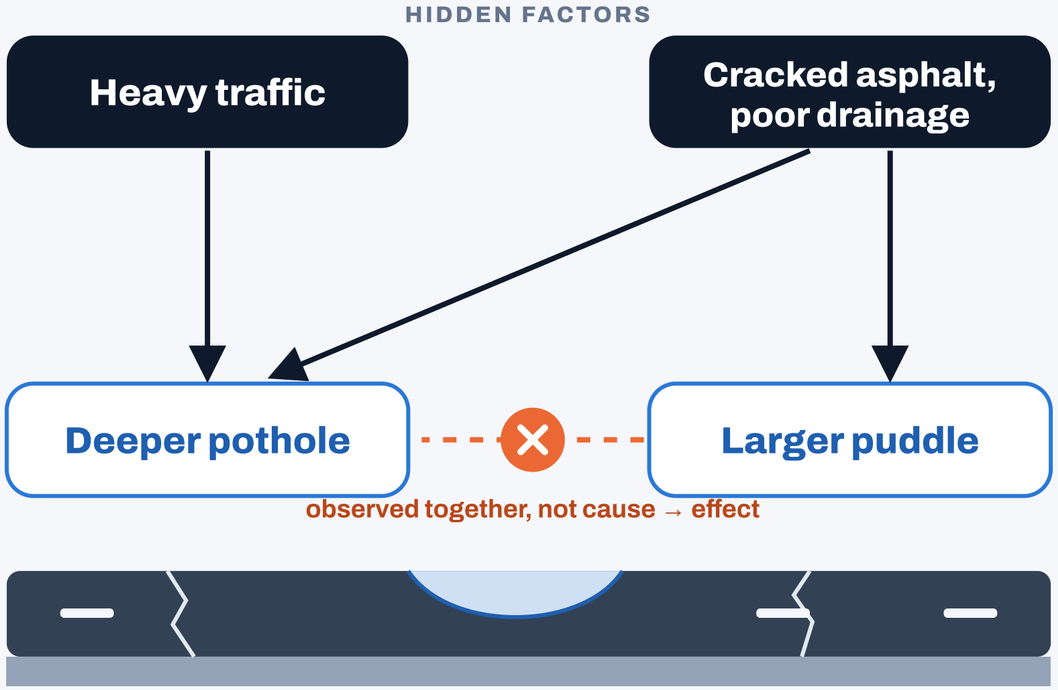


OBSERVED PATTERN

Larger puddles go with deeper potholes.

CORRELATION ≠ CAUSATION · A ROAD-ENGINEERING EXAMPLE

Do puddles cause potholes?



OBSERVED PATTERN

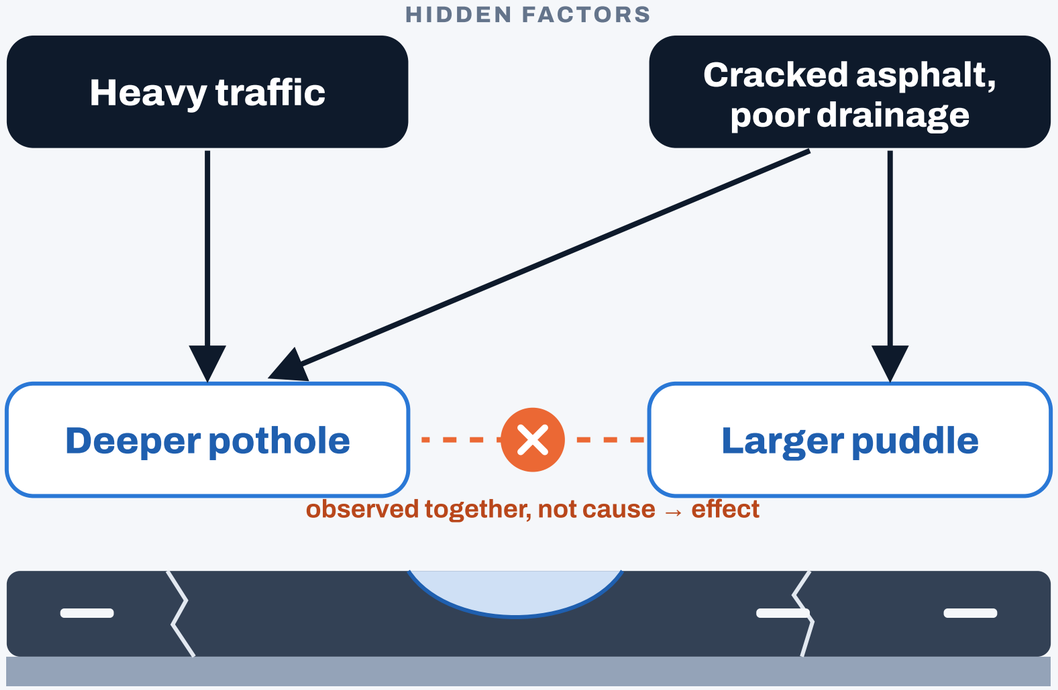
Larger puddles go with deeper potholes.

TEMPTING CONCLUSION

~~**"The puddle caused the pothole."**~~

CORRELATION ≠ CAUSATION · A ROAD-ENGINEERING EXAMPLE

Do puddles cause potholes?



OBSERVED PATTERN

Larger puddles go with deeper potholes.

TEMPTING CONCLUSION

~~**"The puddle caused the pothole."**~~

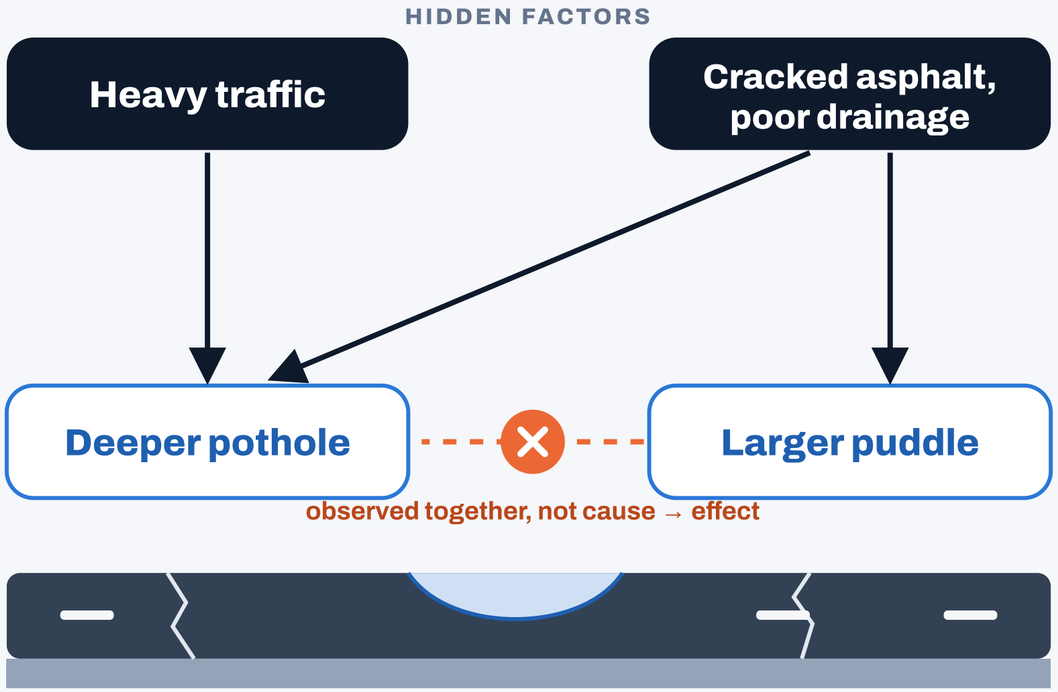
POSSIBLE HIDDEN FACTORS

Traffic, cracked asphalt and depressions shape both.

A correlation points us toward a question. It does not establish the mechanism.

CORRELATION ≠ CAUSATION · A ROAD-ENGINEERING EXAMPLE

Do puddles cause potholes?



OBSERVED PATTERN

Larger puddles go with deeper potholes.

TEMPTING CONCLUSION

~~**"The puddle caused the pothole."**~~

POSSIBLE HIDDEN FACTORS

Traffic, cracked asphalt and depressions shape both.

A correlation points us toward a question. It does not establish the mechanism.

Engineering action should target the underlying cause, not merely the visible association.

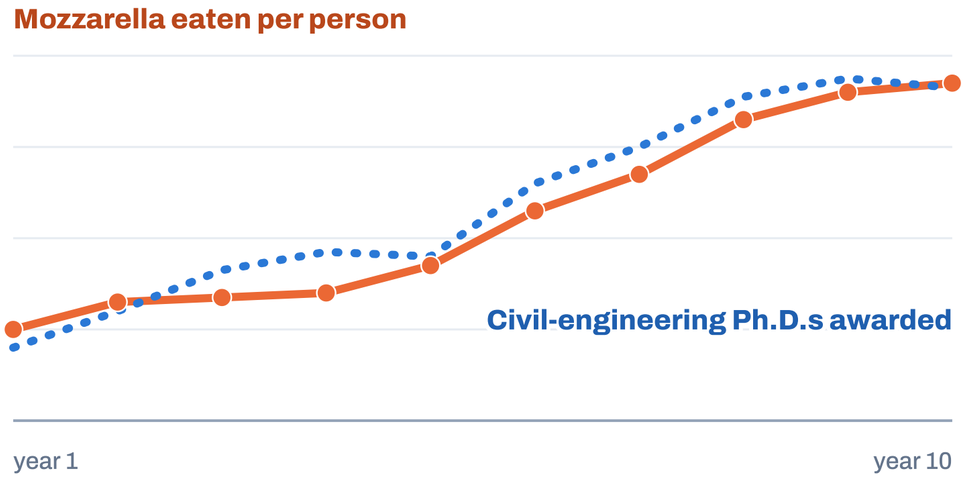
A MEMORABLE COINCIDENCE

Mozzarella and civil-engineering Ph.D.s

Two U.S. series that rose together, 2000–2009

Published correlation: r ≈ 0.95.

Schematic: shape only, not the published values



Vigen, T. *Spurious Correlations* (tylervigen.com), from U.S. Department of Agriculture and National Science Foundation data. See also Science News Explores, “Correlation, causation, coincidence and more”.

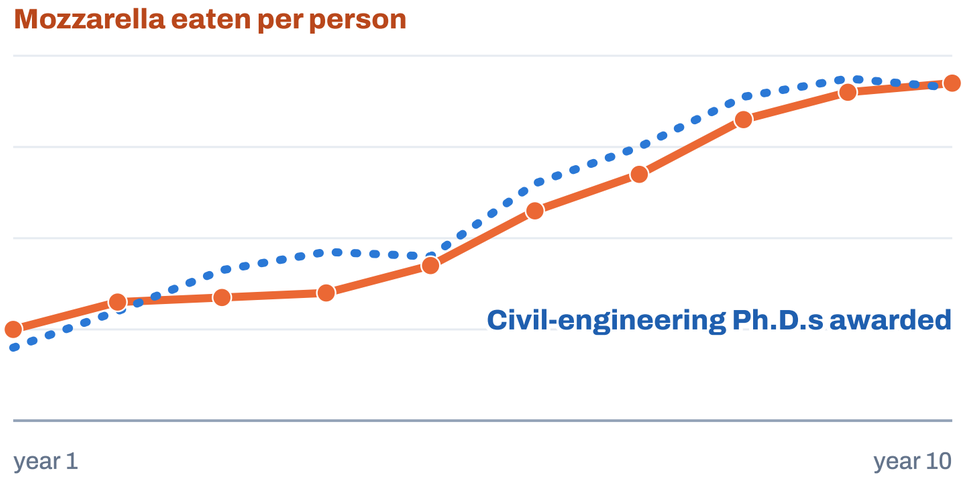
A MEMORABLE COINCIDENCE

Mozzarella and civil-engineering Ph.D.s

Two U.S. series that rose together, 2000–2009

Published correlation: r ≈ 0.95.

Schematic: shape only, not the published values



THE PATTERN

Per-capita mozzarella consumption and civil-engineering doctorates awarded moved together for a decade.

Vigen, T. *Spurious Correlations* (tylervigen.com), from U.S. Department of Agriculture and National Science Foundation data. See also Science News Explores, “Correlation, causation, coincidence and more”.

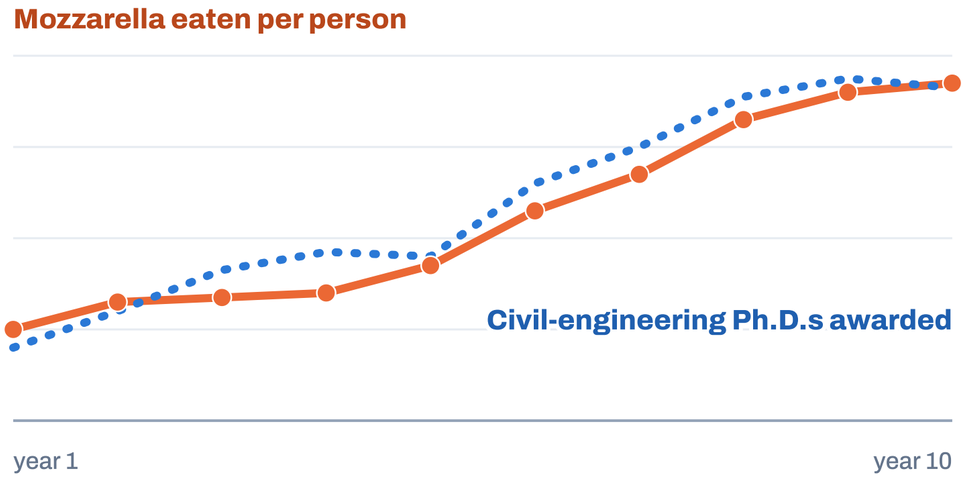
A MEMORABLE COINCIDENCE

Mozzarella and civil-engineering Ph.D.s

Two U.S. series that rose together, 2000–2009

Published correlation: r ≈ 0.95.

Schematic: shape only, not the published values



THE PATTERN

Per-capita mozzarella consumption and civil-engineering doctorates awarded moved together for a decade.

THE LESSON

Strong association can arise by coincidence, or because both follow unrelated upward trends.

Vigen, T. *Spurious Correlations* (tylervigen.com), from U.S. Department of Agriculture and National Science Foundation data. See also Science News Explores, “Correlation, causation, coincidence and more”.

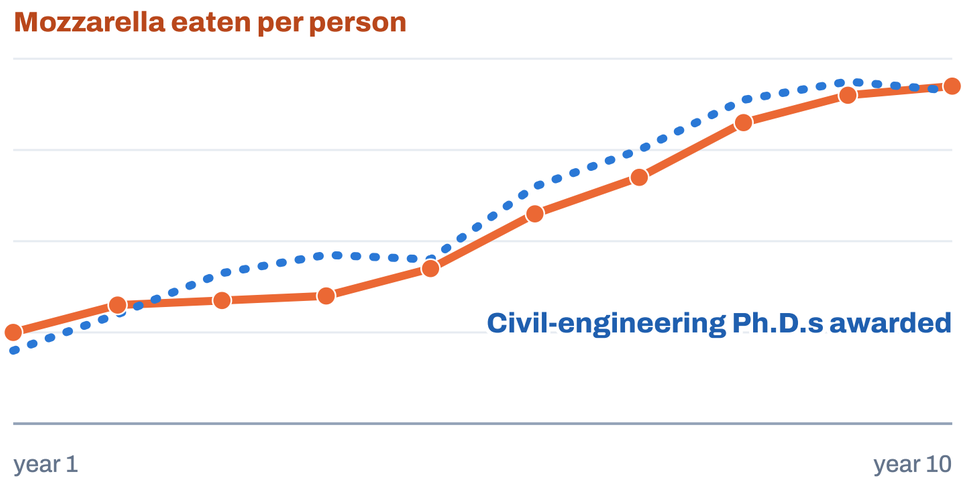
A MEMORABLE COINCIDENCE

Mozzarella and civil-engineering Ph.D.s

Two U.S. series that rose together, 2000–2009

Published correlation: r ≈ 0.95.

Schematic: shape only, not the published values



THE PATTERN

Per-capita mozzarella consumption and civil-engineering doctorates awarded moved together for a decade.

THE LESSON

Strong association can arise by coincidence, or because both follow unrelated upward trends.

Cheese does not cause Ph.D.s.

Vigen, T. *Spurious Correlations* (tylervigen.com), from U.S. Department of Agriculture and National Science Foundation data. See also Science News Explores, “Correlation, causation, coincidence and more”.

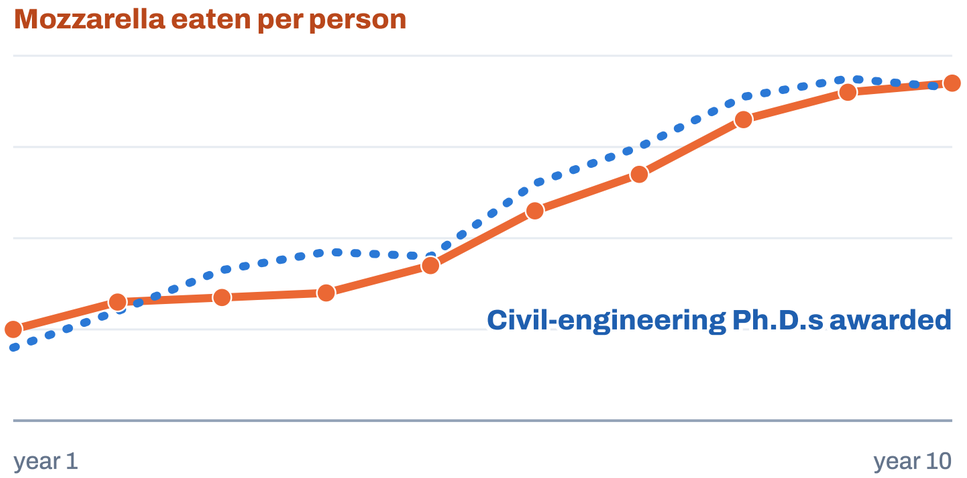
A MEMORABLE COINCIDENCE

Mozzarella and civil-engineering Ph.D.s

Two U.S. series that rose together, 2000–2009

Published correlation: r ≈ 0.95.

Schematic: shape only, not the published values



THE PATTERN

Per-capita mozzarella consumption and civil-engineering doctorates awarded moved together for a decade.

THE LESSON

Strong association can arise by coincidence, or because both follow unrelated upward trends.

Cheese does not cause Ph.D.s.

Use the example to remember the principle, not as evidence of a causal relationship.

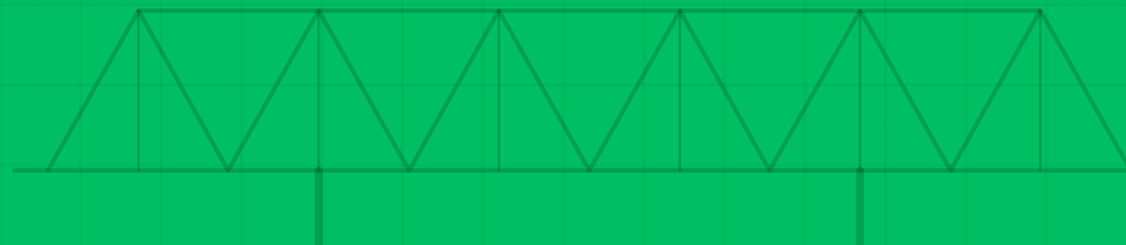
Vigen, T. *Spurious Correlations* (tylervigen.com), from U.S. Department of Agriculture and National Science Foundation data. See also Science News Explores, “Correlation, causation, coincidence and more”.

03

- a Features X and target y
- b Not every column is a feature
- c Data leakage
- d Train ≠ test, and random_state
- e Class balance
- f Scaling: a preview

PART 03 · PREPARATION & EVALUATION SETUP

Prepare the data for an honest model



FORMULATE THE LEARNING PROBLEM

What exactly are we asking the model to learn?

FEATURES · X

8 inputs

- Cement
- Slag
- Fly ash
- Water
- Superplasticizer
- Coarse agg.
- Fine agg.
- Age

FORMULATE THE LEARNING PROBLEM

What exactly are we asking the model to learn?

FEATURES · X

8 inputs

- Cement
- Slag
- Fly ash
- Water
- Superplasticizer
- Coarse agg.
- Fine agg.
- Age





Model

Learns a relationship from examples

FORMULATE THE LEARNING PROBLEM

What exactly are we asking the model to learn?

FEATURES · X

8 inputs

- Cement
- Slag
- Fly ash
- Water
- Superplasticizer
- Coarse agg.
- Fine agg.
- Age

Model

Learns a relationship from examples

TARGET · Y

y

Concrete compressive strength (MPa)

FORMULATE THE LEARNING PROBLEM

What exactly are we asking the model to learn?

FEATURES · X

8 inputs

Cement | Slag | Fly ash | Water | Superplasticizer | Coarse agg. | Fine agg. | Age

Model

Learns a relationship from examples

TARGET · Y

y

Concrete compressive strength (MPa)

Session02_data_preparation.ipynb X, y

```
X = df.drop(columns="strength")   # features
y = df["strength"]                  # target
X.shape, y.shape
```

OUTPUT

```
((1030, 8), (1030,))
```

FORMULATE THE LEARNING PROBLEM

What exactly are we asking the model to learn?

FEATURES · X

8 inputs

Cement | Slag | Fly ash | Water | Superplasticizer | Coarse agg. | Fine agg. | Age

→

Model

Learns a relationship from examples

→

TARGET · Y

y

Concrete compressive strength (MPa)

Session02_data_preparation.ipynb X, y

```
X = df.drop(columns="strength")   # features
y = df["strength"]                  # target
X.shape, y.shape
```

OUTPUT

```
((1030, 8), (1030,))
```

READ THE OUTPUT

`(1030, 8)`	1,030 rows, 8 feature columns: the table ***X***
`(1030,)`	one strength value per row: the vector ***y***

FORMULATE THE LEARNING PROBLEM

What exactly are we asking the model to learn?

FEATURES · X

8 inputs

Cement | Slag | Fly ash | Water | Superplasticizer | Coarse agg. | Fine agg. | Age

→

Model

Learns a relationship from examples

→

TARGET · Y

y

Concrete compressive strength (MPa)

Session02_data_preparation.ipynb X, y

```
X = df.drop(columns="strength")   # features
y = df["strength"]                  # target
X.shape, y.shape
```

OUTPUT

```
((1030, 8), (1030,))
```

READ THE OUTPUT

`(1030, 8)`	1,030 rows, 8 feature columns: the table ***X***
`(1030,)`	one strength value per row: the vector ***y***

Features are the information we use. The target is the quantity we want to predict.

COLUMN ROLES

Not every column should become a feature

COLUMN ROLES

Not every column should become a feature

FEATURE

Known at prediction time

Information we will genuinely have.

`cement, water, age`

COLUMN ROLES

Not every column should become a feature

FEATURE

Known at prediction time

Information we will genuinely have.

`cement, water, age`

TARGET

What we predict

One column, chosen deliberately.

`strength · fault class`

COLUMN ROLES

Not every column should become a feature

FEATURE

Known at prediction time

Information we will genuinely have.

`cement, water, age`

TARGET

What we predict

One column, chosen deliberately.

`strength · fault class`

IDENTIFIER

Names, IDs, dates

Labels for a row; often no predictive meaning.

`"D-1/3/90" · specimen no.`

COLUMN ROLES

Not every column should become a feature

FEATURE

Known at prediction time

Information we will genuinely have.

`cement, water, age`

TARGET

What we predict

One column, chosen deliberately.

`strength · fault class`

IDENTIFIER

Names, IDs, dates

Labels for a row; often no predictive meaning.

`"D-1/3/90" · specimen no.`

LEAKED INFORMATION

Not available in time

Would not exist when the prediction is made.

`56-day strength → 28-day`

COLUMN ROLES

Not every column should become a feature

FEATURE

Known at prediction time

Information we will genuinely have.

`cement, water, age`

TARGET

What we predict

One column, chosen deliberately.

`strength · fault class`

IDENTIFIER

Names, IDs, dates

Labels for a row; often no predictive meaning.

`"D-1/3/90" · specimen no.`

LEAKED INFORMATION

Not available in time

Would not exist when the prediction is made.

`56-day strength → 28-day`

BEFORE USING A COLUMN, ASK

01 What does it represent?

02 When would it be known?

03 Could it reveal the answer indirectly?

COLUMN ROLES

Not every column should become a feature

FEATURE

Known at prediction time

Information we will genuinely have.

`cement, water, age`

TARGET

What we predict

One column, chosen deliberately.

`strength · fault class`

IDENTIFIER

Names, IDs, dates

Labels for a row; often no predictive meaning.

`"D-1/3/90" · specimen no.`

LEAKED INFORMATION

Not available in time

Would not exist when the prediction is made.

`56-day strength → 28-day`

BEFORE USING A COLUMN, ASK

01 What does it represent?

02 When would it be known?

03 Could it reveal the answer indirectly?

“I have a column” does not mean “I should give it to the model.”

THE SILENT KILLER

Data leakage: excellent tests, disappointing reality

01 Information unavailable at prediction time

THE SILENT KILLER

Data leakage: excellent tests, disappointing reality

01 Information unavailable at prediction time

↓

02 Influences training or preprocessing

THE SILENT KILLER

Data leakage: excellent tests, disappointing reality

01 Information unavailable at prediction time

↓

02 Influences training or preprocessing

↓

03 Test performance looks excellent

THE SILENT KILLER

Data leakage: excellent tests, disappointing reality

01 Information unavailable at prediction time

↓

02 Influences training or preprocessing

↓

03 Test performance looks excellent

↓

04 Real-world performance disappoints

THE SILENT KILLER

Data leakage: excellent tests, disappointing reality

01 Information unavailable at prediction time

↓

02 Influences training or preprocessing

↓

03 Test performance looks excellent

↓

04 Real-world performance disappoints

LEAK THROUGH A COLUMN

Predicting 28-day strength at the mix-design stage

...with the 56-day result or an acceptance-test pass/fail as an input. Neither exists when the mix is designed.

THE SILENT KILLER

Data leakage: excellent tests, disappointing reality

01 Information unavailable at prediction time

↓

02 Influences training or preprocessing

↓

03 Test performance looks excellent

↓

04 Real-world performance disappoints

LEAK THROUGH A COLUMN

Predicting 28-day strength at the mix-design stage

…with the 56-day result or an acceptance-test pass/fail as an input. Neither exists when the mix is designed.

LEAK THROUGH PREPROCESSING

Filling or scaling with the full dataset

Medians or scales computed before the split let the test rows quietly shape training.

```
split first → fit on train → apply to test
```

THE SILENT KILLER

Data leakage: excellent tests, disappointing reality

01 Information unavailable at prediction time

↓

02 Influences training or preprocessing

↓

03 Test performance looks excellent

↓

04 Real-world performance disappoints

LEAK THROUGH A COLUMN

Predicting 28-day strength at the mix-design stage

…with the 56-day result or an acceptance-test pass/fail as an input. Neither exists when the mix is designed.

LEAK THROUGH PREPROCESSING

Filling or scaling with the full dataset

Medians or scales computed before the split let the test rows quietly shape training.

```
split first → fit on train → apply to test
```

Ask: would this information genuinely be available when I make the prediction?

EVALUATION SETUP · SPLIT BEFORE YOU FIT

Train ≠ Test

Full dataset · all available observations 1,030 mixes

EVALUATION SETUP · SPLIT BEFORE YOU FIT

Train ≠ Test

Full dataset · all available observations `1,030 mixes`

↓ SPLIT THE DATA BEFORE FITTING THE MODEL

TRAIN · 80%
The model learns from these observations.
`824 mixes`

TEST · 20%
Kept aside for unseen data.
`206 mixes`

EVALUATION SETUP · SPLIT BEFORE YOU FIT

Train ≠ Test

Full dataset · all available observations `1,030 mixes`

↓ SPLIT THE DATA BEFORE FITTING THE MODEL

TRAIN · 80%
The model learns from these observations.
`824 mixes`

TEST · 20%
Kept aside for unseen data.
`206 mixes`

Session02_data_preparation.ipynb — split

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

X_TRAIN.SHAPE, X_TEST.SHAPE

```
((824, 8), (206, 8))
```

EVALUATION SETUP · SPLIT BEFORE YOU FIT

Train ≠ Test

Full dataset · all available observations 1,030 mixes

SPLIT THE DATA BEFORE FITTING THE MODEL

TRAIN · 80%
The model learns from these observations.
824 mixes

TEST · 20%
Kept aside for unseen data.
206 mixes

Session02_data_preparation.ipynb split

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

X_TRAIN.SHAPE, X_TEST.SHAPE

```
((824, 8), (206, 8))
```

REMEMBER THE 25 DUPLICATES?

Remove them before splitting.

A row in train with its twin in test makes the test look easier than reality.

WHY KEEP A TEST SET AT ALL?

Why not evaluate on the data the model has seen?

WHY KEEP A TEST SET AT ALL?

Why not evaluate on the data the model has seen?

Seen data

A model may perform well on examples it has already learned from, even by memorising them.

Like a student scoring full marks on last year’s paper after memorising its answers.

WHY KEEP A TEST SET AT ALL?

Why not evaluate on the data the model has seen?

Seen data

A model may perform well on examples it has already learned from, even by memorising them.

Like a student scoring full marks on last year's paper after memorising its answers.

Unseen data

We need an honest estimate of how it may perform on new observations: the next mix, the next plate.

Like a new paper: it shows what was understood, not what was remembered.

WHY KEEP A TEST SET AT ALL?

Why not evaluate on the data the model has seen?

Seen data

A model may perform well on examples it has already learned from, even by memorising them.

Like a student scoring full marks on last year's paper after memorising its answers.

Unseen data

We need an honest estimate of how it may perform on new observations: the next mix, the next plate.

Like a new paper: it shows what was understood, not what was remembered.

IN SESSION 03 — **Fit** on train → **Predict** the test rows → **Compare** with their true *y* → **An honest error estimate**

WHY KEEP A TEST SET AT ALL?

Why not evaluate on the data the model has seen?

Seen data

A model may perform well on examples it has already learned from, even by memorising them.

Like a student scoring full marks on last year's paper after memorising its answers.

Unseen data

We need an honest estimate of how it may perform on new observations: the next mix, the next plate.

Like a new paper: it shows what was understood, not what was remembered.

IN SESSION 03 — **Fit** on train → **Predict** the test rows → **Compare** with their true *y* → **An honest error estimate**

Training performance ≠ generalisation performance.

REPRODUCIBILITY · WHICH 8 OF 40 ROWS LAND IN THE TEST SET?

Why random_state = 42?

train row test row

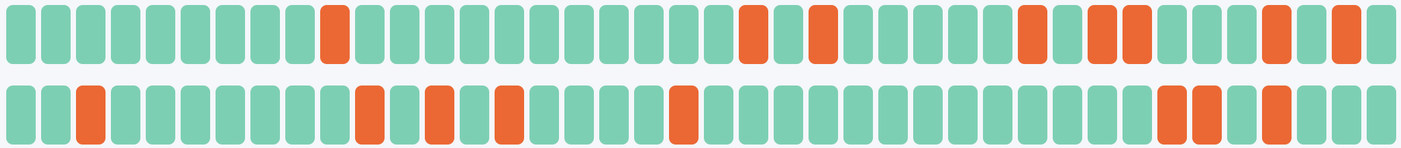
REPRODUCIBILITY · WHICH 8 OF 40 ROWS LAND IN THE TEST SET?

Why random_state = 42?

train row test row

No fixed seed

Results may vary because the samples differ.



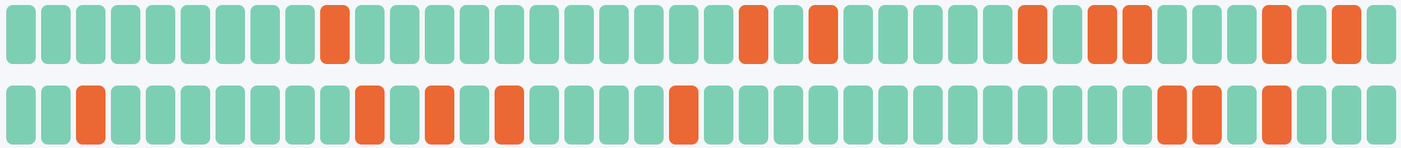
REPRODUCIBILITY · WHICH 8 OF 40 ROWS LAND IN THE TEST SET?

Why random_state = 42?

train row test row

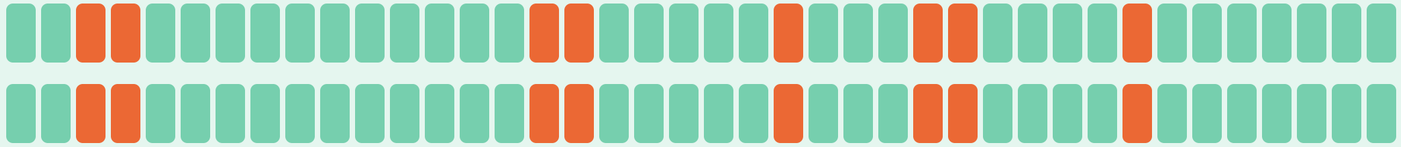
No fixed seed

Results may vary because the samples differ.



random_state=42

Same split every time: easier to reproduce and compare.



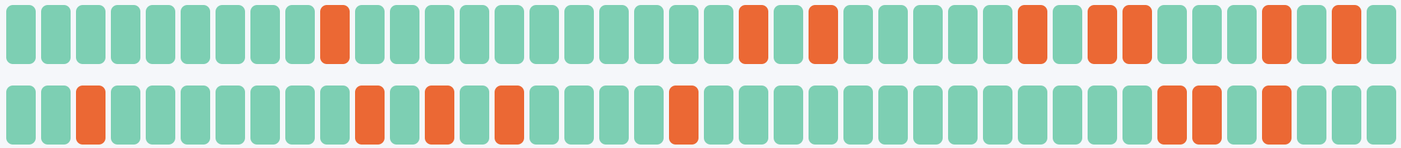
REPRODUCIBILITY · WHICH 8 OF 40 ROWS LAND IN THE TEST SET?

Why random_state = 42?

train row test row

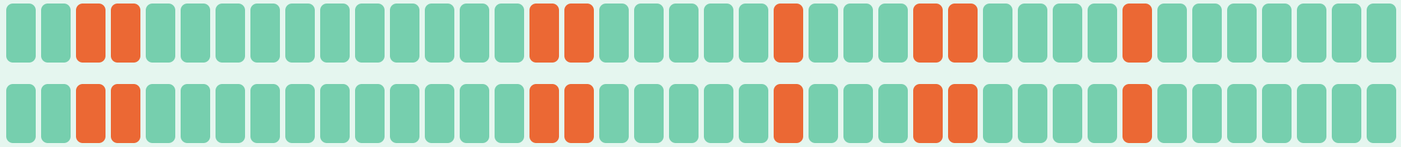
No fixed seed

Results may vary because the samples differ.



random_state=42

Same split every time: easier to reproduce and compare.



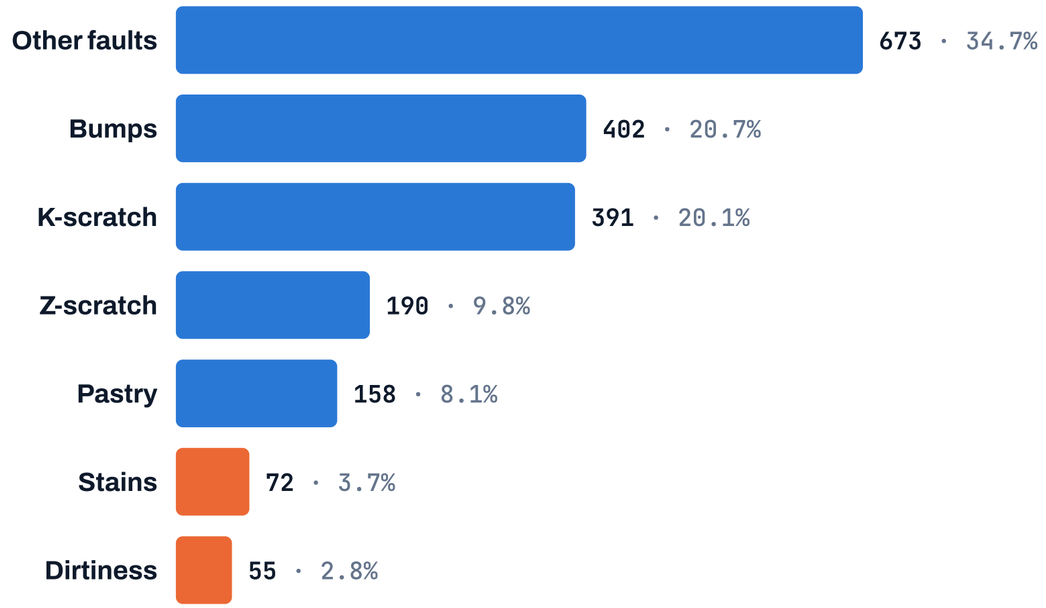
42 has no engineering meaning. It is simply a fixed random seed. Any number works, as long as it is recorded.

STEEL PLATES · WHAT DOES THE TARGET LOOK LIKE?

Class balance changes what “good” means

1,941 plates, 7 fault classes

y = fault type. Orange: classes below 5% of the plates.



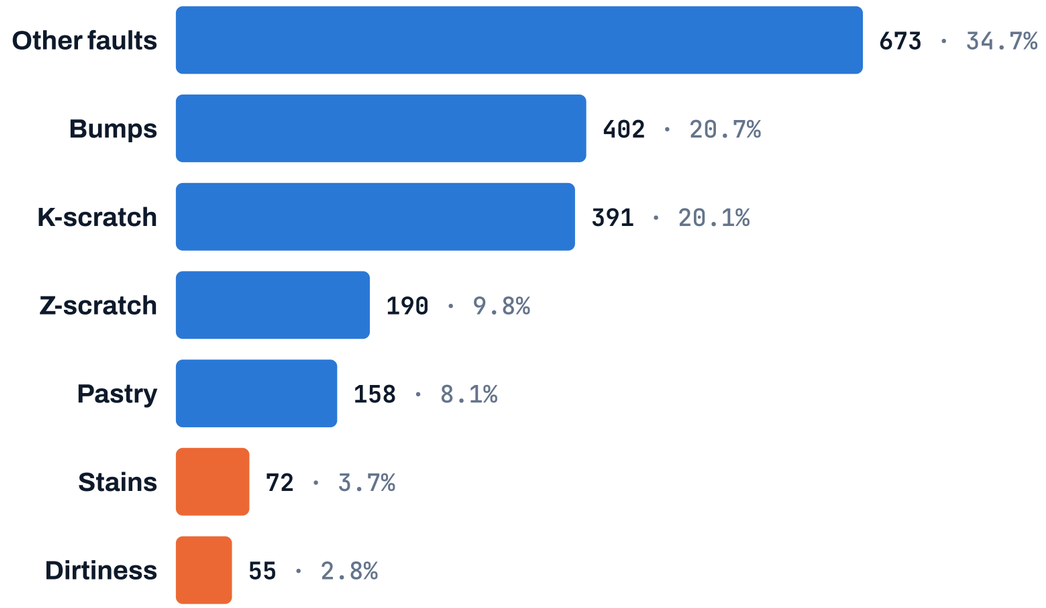
Data: Steel Plates Faults, UCI Machine Learning Repository (Buscema, Terzi & Tastle, 2010). Every plate carries exactly one fault label.

STEEL PLATES · WHAT DOES THE TARGET LOOK LIKE?

Class balance changes what “good” means

1,941 plates, 7 fault classes

y = fault type. Orange: classes below 5% of the plates.



34.7%

accuracy of a useless model that always answers “Other faults”.

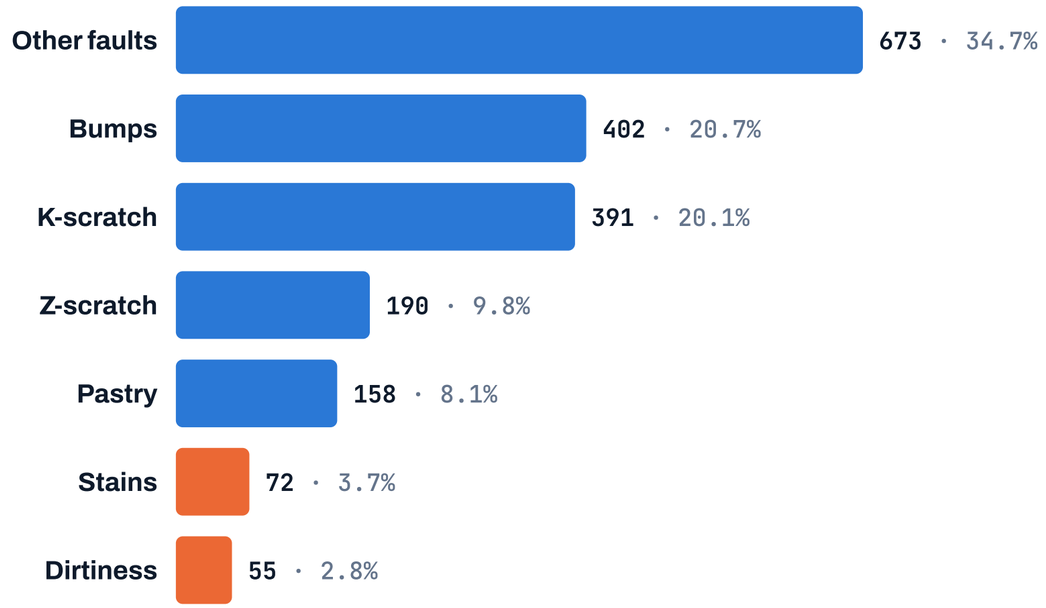
Data: Steel Plates Faults, UCI Machine Learning Repository (Buscema, Terzi & Tastle, 2010). Every plate carries exactly one fault label.

STEEL PLATES · WHAT DOES THE TARGET LOOK LIKE?

Class balance changes what “good” means

1,941 plates, 7 fault classes

y = fault type. Orange: classes below 5% of the plates.



34.7%

accuracy of a useless model that always answers “Other faults”.

RARE CLASSES

A random 20% test set holds only about 11 Dirtiness plates.

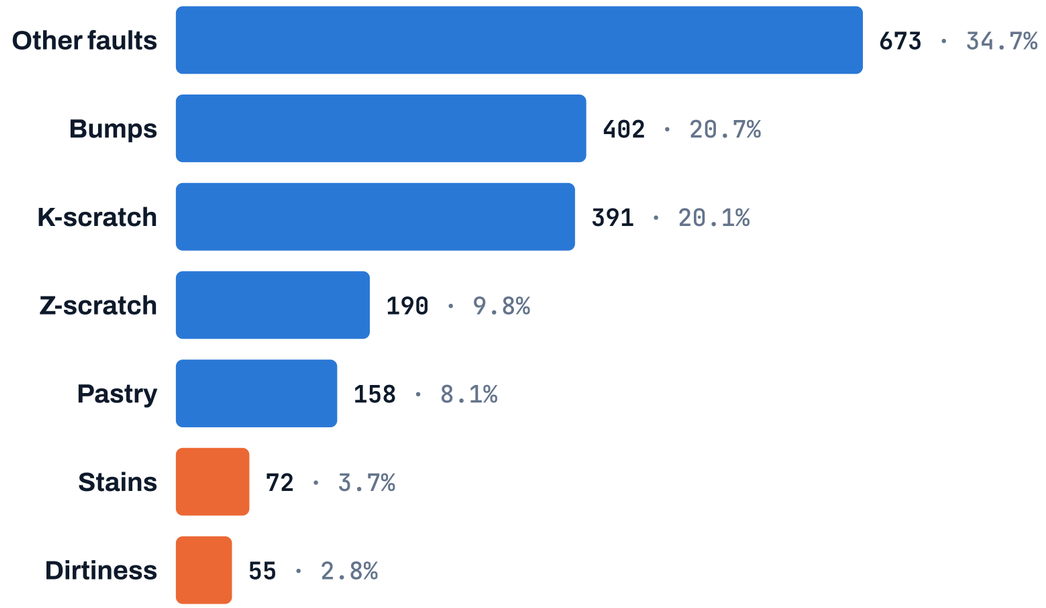
Data: Steel Plates Faults, UCI Machine Learning Repository (Buscema, Terzi & Tastle, 2010). Every plate carries exactly one fault label.

STEEL PLATES · WHAT DOES THE TARGET LOOK LIKE?

Class balance changes what “good” means

1,941 plates, 7 fault classes

y = fault type. Orange: classes below 5% of the plates.



34.7%

accuracy of a useless model that always answers “Other faults”.

RARE CLASSES

A random 20% test set holds only about 11 Dirtiness plates.

KEEP THE PROPORTIONS

Stratify the split by the target:

```
train_test_split(…, stratify=y)
```

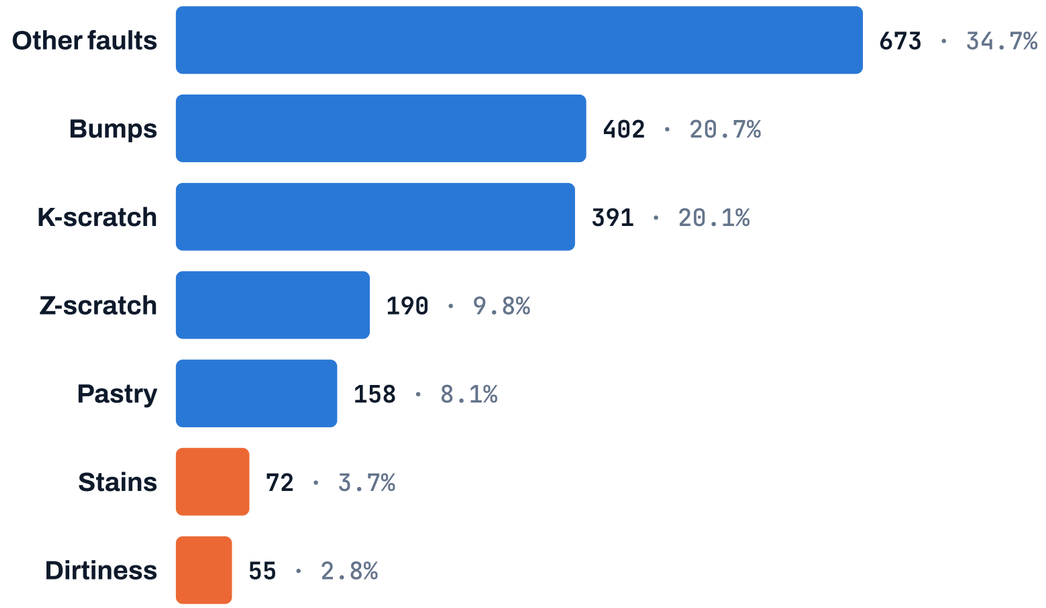
Data: Steel Plates Faults, UCI Machine Learning Repository (Buscema, Terzi & Tastle, 2010). Every plate carries exactly one fault label.

STEEL PLATES · WHAT DOES THE TARGET LOOK LIKE?

Class balance changes what “good” means

1,941 plates, 7 fault classes

y = fault type. Orange: classes below 5% of the plates.



34.7%

accuracy of a useless model that always answers “Other faults”.

RARE CLASSES

A random 20% test set holds only about 11 Dirtiness plates.

KEEP THE PROPORTIONS

Stratify the split by the target:

```
train_test_split(…, stratify=y)
```

Before building a classifier, look at the target. Balance decides which accuracy is impressive. (Session 05)

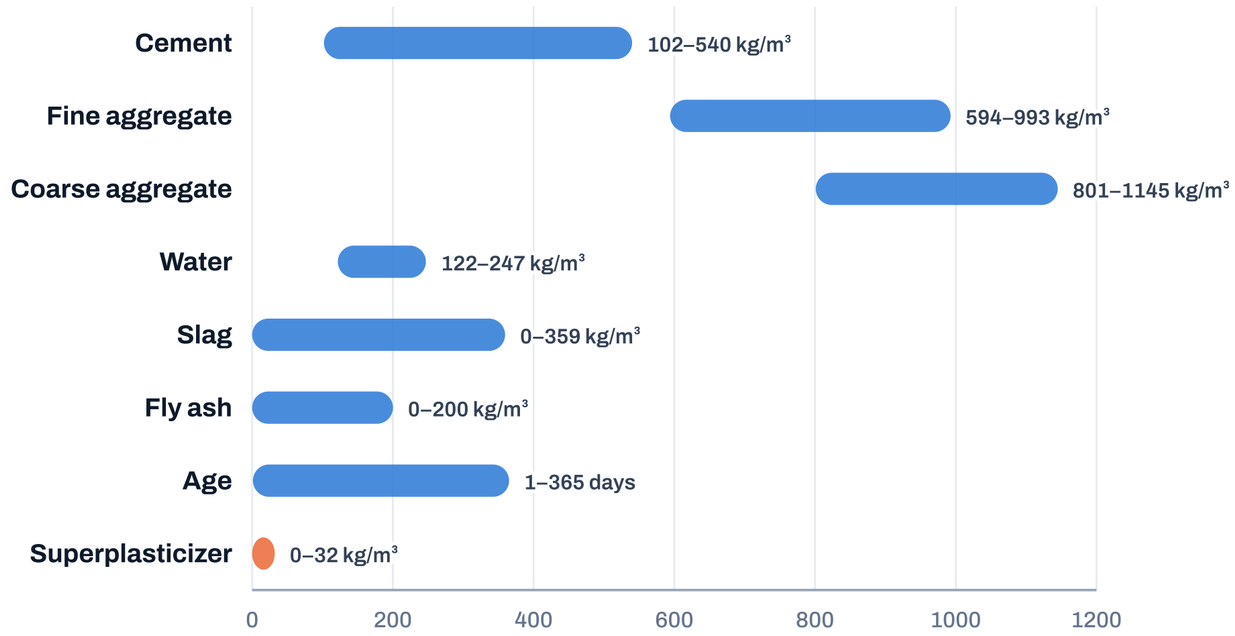
Data: Steel Plates Faults, UCI Machine Learning Repository (Buscema, Terzi & Tastle, 2010). Every plate carries exactly one fault label.

AN IMPORTANT PREVIEW

Scaling: same data, very different numbers

Range of each concrete input, on one axis

Aggregates sit in the hundreds of kg/m³; superplasticizer in the tens; age in days.



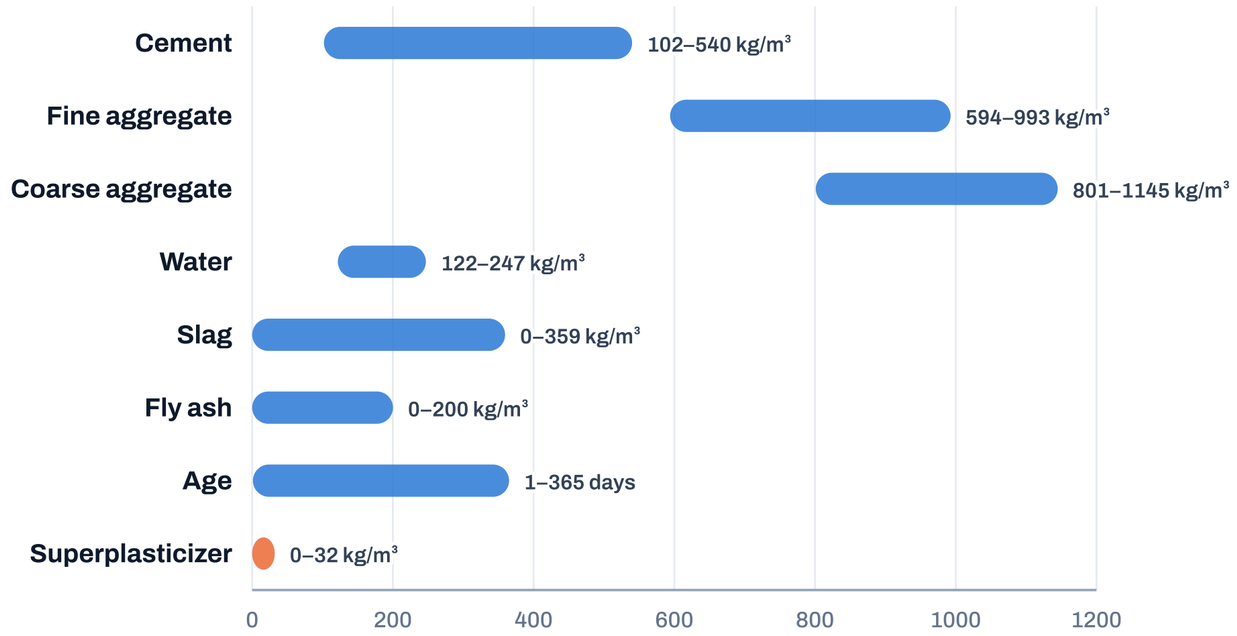
Data: UCI Concrete Compressive Strength (Yeh, 1998). Bars run from the minimum to the maximum in the dataset.

AN IMPORTANT PREVIEW

Scaling: same data, very different numbers

Range of each concrete input, on one axis

Aggregates sit in the hundreds of kg/m³; superplasticizer in the tens; age in days.



Sensitive to scale

Distance- and gradient-based methods: k-nearest neighbours, SVMs, neural networks.

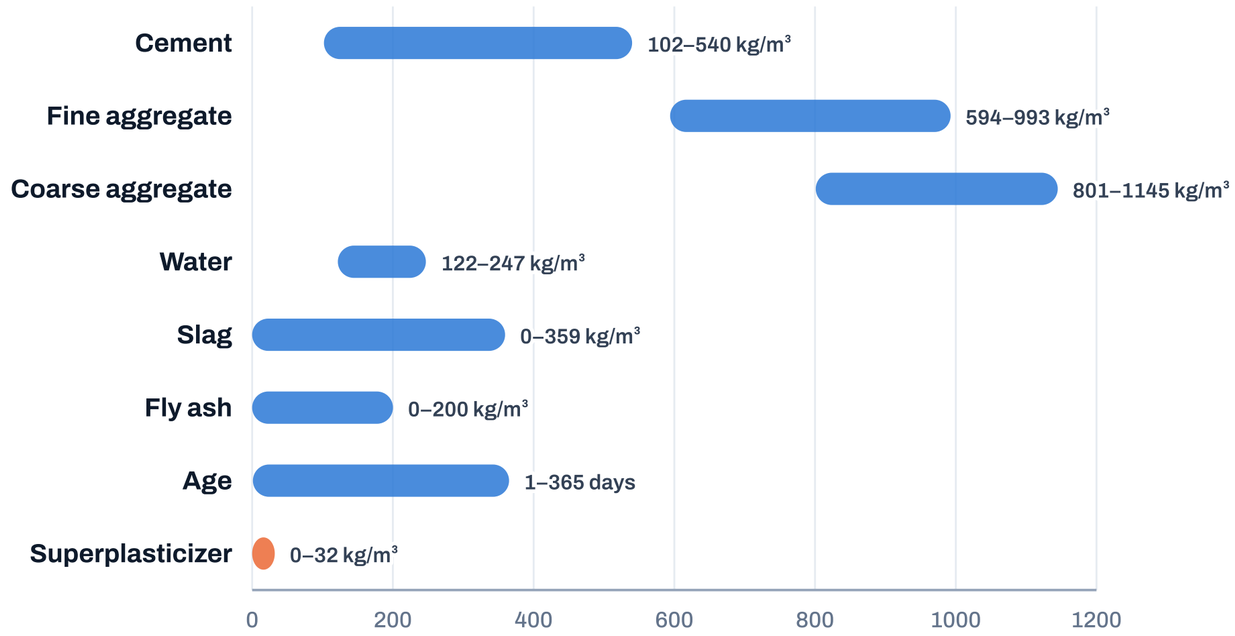
Data: UCI Concrete Compressive Strength (Yeh, 1998). Bars run from the minimum to the maximum in the dataset.

AN IMPORTANT PREVIEW

Scaling: same data, very different numbers

Range of each concrete input, on one axis

Aggregates sit in the hundreds of kg/m³; superplasticizer in the tens; age in days.



Sensitive to scale

Distance- and gradient-based methods: k-nearest neighbours, SVMs, neural networks.

Less sensitive

Tree-based methods: decision trees, random forests (Session 04).

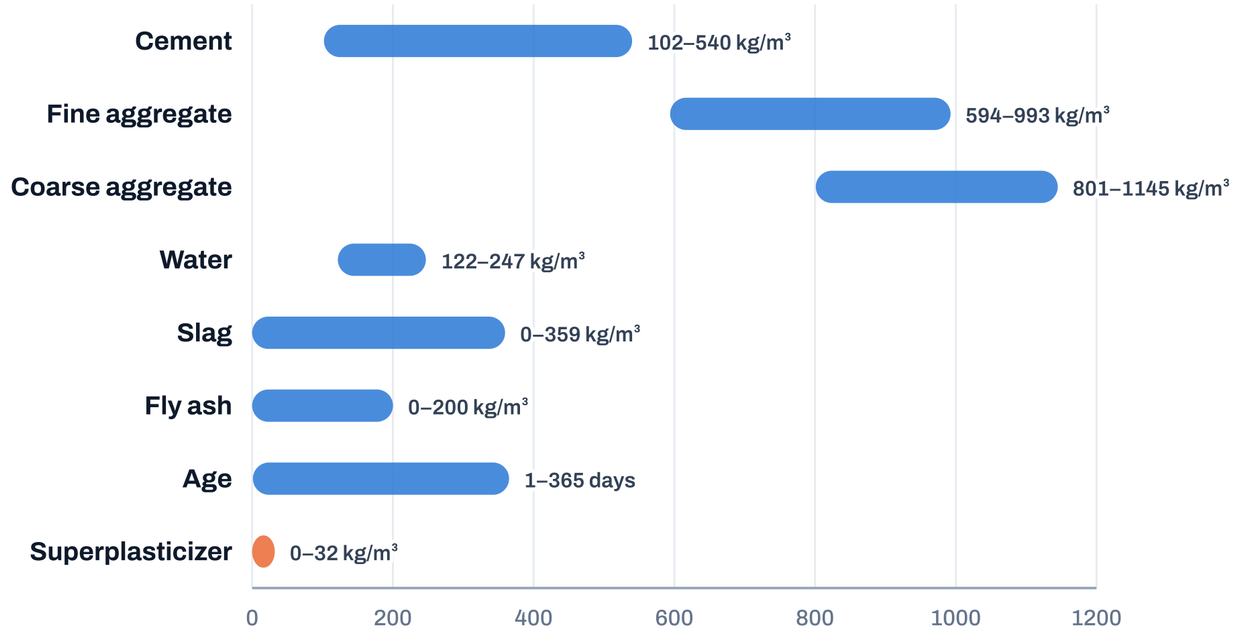
Data: UCI Concrete Compressive Strength (Yeh, 1998). Bars run from the minimum to the maximum in the dataset.

AN IMPORTANT PREVIEW

Scaling: same data, very different numbers

Range of each concrete input, on one axis

Aggregates sit in the hundreds of kg/m³; superplasticizer in the tens; age in days.



Sensitive to scale

Distance- and gradient-based methods: k-nearest neighbours, SVMs, neural networks.

Less sensitive

Tree-based methods: decision trees, random forests (Session 04).

THE RULE

Fit preprocessing on training data, not on the full dataset.

```
scaler.fit(X_train)      # learn
scaler.transform(X_test) # apply
```

Data: UCI Concrete Compressive Strength (Yeh, 1998). Bars run from the minimum to the maximum in the dataset.

SUMMARY · TODAY'S WORKFLOW

The complete journey

SUMMARY · TODAY'S WORKFLOW

The complete journey

01

Understand the variables

`data dictionary`

02

Check data quality

`shape · dtypes`

03

Investigate missing values & duplicates

`isna · duplicated`

04

Check physical plausibility

`describe`

SUMMARY · TODAY'S WORKFLOW

The complete journey

01

Understand the variables

`data dictionary`

02

Check data quality

`shape · dtypes`

03

Investigate missing values & duplicates

`isna · duplicated`

04

Check physical plausibility

`describe`

05

Explore distributions & relationships

`hist · corr`

06

Define features X and target y

`drop · select`

07

Prevent leakage

`known in time?`

08

Split train / test

`train_test_split`

SUMMARY · TODAY'S WORKFLOW

The complete journey

01

Understand the variables

`data dictionary`

02

Check data quality

`shape · dtypes`

03

Investigate missing values & duplicates

`isna · duplicated`

04

Check physical plausibility

`describe`

05

Explore distributions & relationships

`hist · corr`

06

Define features X and target y

`drop · select`

07

Prevent leakage

`known in time?`

08

Split train / test

`train_test_split`

Good ML begins before the algorithm.

HANDS-ON · IN COLAB

Today’s notebook, section by section

HANDS-ON · IN COLAB

Today’s notebook, section by section

01	**Audit the concrete data** Size, types, missing values, duplicates, ranges	`df.describe()`

HANDS-ON · IN COLAB

Today’s notebook, section by section

01	**Audit the concrete data** Size, types, missing values, duplicates, ranges	`df.describe()`
02	**Read the water-treatment file** Turn “?” into NaN, then count and map the gaps	`na_values="?"`

HANDS-ON · IN COLAB

Today’s notebook, section by section

01	**Audit the concrete data** Size, types, missing values, duplicates, ranges	`df.describe()`
02	**Read the water-treatment file** Turn “?” into NaN, then count and map the gaps	`na_values="?"`
03	**Explore the concrete data** Histograms, box plots, correlation with strength	`df.hist()`

HANDS-ON · IN COLAB

Today’s notebook, section by section

01	**Audit the concrete data** Size, types, missing values, duplicates, ranges	`df.describe()`
02	**Read the water-treatment file** Turn “?” into NaN, then count and map the gaps	`na_values="?"`
03	**Explore the concrete data** Histograms, box plots, correlation with strength	`df.hist()`
04	**Prepare the steel-plate data** Class counts, X and y, a stratified train/test split	`stratify=y`

HANDS-ON · IN COLAB

Today’s notebook, section by section

01	**Audit the concrete data** Size, types, missing values, duplicates, ranges	`df.describe()`
02	**Read the water-treatment file** Turn “?” into NaN, then count and map the gaps	`na_values="?"`
03	**Explore the concrete data** Histograms, box plots, correlation with strength	`df.hist()`
04	**Prepare the steel-plate data** Class counts, X and y, a stratified train/test split	`stratify=y`

WORKSHOP NOTEBOOK LINK

Open today’s notebook and save your own copy.

Run the cells in order. Read every output before moving on.

Link shared in the session chat

THE FINAL TAKEAWAY

Before asking “Which ML algorithm should I use?”

THE FINAL TAKEAWAY

Before asking “Which ML algorithm should I use?”

ASK

“Do I understand my data?”

THE FINAL TAKEAWAY

Before asking “Which ML algorithm should I use?”

ASK **“Do I understand my data?”**

AND **“Would this be available at prediction time?”**

THE FINAL TAKEAWAY

Before asking “Which ML algorithm should I use?”

ASK **“Do I understand my data?”**

AND **“Would this be available at prediction time?”**

Understand the data → Formulate the problem → Protect the evaluation → Then model

TODAY → NEXT TIME

Are we ready to build the model?

TODAY · SESSION 02

We learned to inspect, question and prepare engineering data.

- Audit, check plausibility, map missing values
- Investigate outliers before deleting anything
- Choose X and y, prevent leakage, split train/test

TODAY → NEXT TIME

Are we ready to build the model?

TODAY · SESSION 02

We learned to inspect, question and prepare engineering data.

- Audit, check plausibility, map missing values
- Investigate outliers before deleting anything
- Choose X and y, prevent leakage, split train/test

NEXT · SESSION 03 · YOUR FIRST ENGINEERING PREDICTION MODEL

Can linear regression predict concrete strength well? How will we judge it?

- Fit a model on the training set we made today
- Judge it on the test set it has never seen

`MAE` `RMSE` `$R^2$` `residuals` `cross-validation`

BEFORE SESSION 03

Three small things to try

BEFORE SESSION 03

Three small things to try

1

Finish the audit

Run today’s notebook end to end. Remove the duplicates and note how many rows remain.

```
df.drop_duplicates()
```

BEFORE SESSION 03

Three small things to try

1

Finish the audit

Run today’s notebook end to end. Remove the duplicates and note how many rows remain.

```
df.drop_duplicates()
```

2

Write a dictionary

For a dataset from your own field: meaning, unit and role (X, y, ID) of every column.

```
column · meaning · unit · role
```

BEFORE SESSION 03

Three small things to try

1

Finish the audit

Run today’s notebook end to end. Remove the duplicates and note how many rows remain.

```
df.drop_duplicates()
```

2

Write a dictionary

For a dataset from your own field: meaning, unit and role (X, y, ID) of every column.

```
column · meaning · unit · role
```

3

Hunt for one leak

Name a column in your field that would look predictive but is only known after the fact.

```
known at prediction time?
```

BEFORE SESSION 03

Three small things to try

1

Finish the audit

Run today’s notebook end to end. Remove the duplicates and note how many rows remain.

```
df.drop_duplicates()
```

2

Write a dictionary

For a dataset from your own field: meaning, unit and role (X, y, ID) of every column.

```
column · meaning · unit · role
```

3

Hunt for one leak

Name a column in your field that would look predictive but is only known after the fact.

```
known at prediction time?
```

03 **Your First Engineering Prediction Model**

Linear regression, error metrics and cross-validation on concrete.

Questions

Unmute, or drop your question in the chat.



- Data quality
- Missing values & outliers
- Leakage & splitting
- Your own engineering data

Dr. Abdul Waris
Research Associate · Universität Siegen, Germany

END OF SESSION 02

Thank you.

See you in session 03: **Your First Engineering Prediction Model.**

Dr. Abdul Waris
Research Associate · Universität Siegen, Germany

Course page
qudeltastudios.com/courses

QUDELTA: STUDIOS | Technology partner